

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Procedurální generování dungeonů ve 2D hrách

Implementace a srovnání algoritmů v herním enginu Godot



2026

Vedoucí práce:
doc. RNDr. Jan Konečný, Ph.D.

Jesse Sadowý

Studijní program: Informační technologie,
kombinovaná forma

Bibliografické údaje

Autor: Jesse Sadowý

Název práce: Procedurální generování dungeonů ve 2D hrách (Implementace a srovnání algoritmů v herním enginu Godot)

Typ práce: bakalářská práce

Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci

Rok obhajoby: 2026

Studijní program: Informační technologie, kombinovaná forma

Vedoucí práce: doc. RNDr. Jan Konečný, Ph.D.

Počet stran: 52

Přílohy: zdrojový kód aplikace

Jazyk práce: český

Bibliographic info

Author: Jesse Sadowý

Title: Procedural Generation of Dungeons in 2D Games (Implementation and Comparison of Algorithms in Godot Game Engine)

Thesis type: bachelor thesis

Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc

Year of defense: 2026

Study program: Information Technologies, combined form

Supervisor: doc. RNDr. Jan Konečný, Ph.D.

Page count: 52

Supplements: application source code

Thesis language: Czech

Anotace

Bakalářská práce se zabývá procedurálním generováním dungeonových map ve 2D hrách. Jsou popsány základní principy procedurálního generování obsahu a vybrané metody tvorby dungeonových struktur. V herním engineu Godot bylo implementováno a experimentálně porovnáno pět generovacích algoritmů z hlediska času generování a pokrytí mapy průchozím prostorem. Výsledky ukazují výrazné rozdíly mezi metodami a práce diskutuje vhodnost jednotlivých přístupů pro různé typy herního prostředí.

Synopsis

This bachelor's thesis deals with procedural generation of dungeon maps in 2D games. It describes the basic principles of procedural content generation and selected methods for creating dungeon structures. Five generation algorithms were implemented and experimentally compared in the Godot game engine in terms of generation time and floor coverage. The results show significant differences between the methods, and the thesis discusses the suitability of each approach for different types of game environments.

Klíčová slova: procedurální generování; dungeon; 2D hry; Godot; herní engine; algoritmus

Keywords: procedural generation; dungeon; 2D games; Godot; game engine; algorithm

Děkuji vedoucímu práce doc. RNDr. Janu Konečnému, Ph.D. za odborné vedení a cenné připomínky při vypracování této práce.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Úvod	9
2	Současný stav řešené problematiky	10
2.1	Procedurální generování obsahu ve hrách	10
2.1.1	Vymezení a definice pojmu	10
2.1.2	Výhody a nevýhody procedurálního generování	10
2.1.3	Přístupy k procedurálnímu generování	11
2.2	Historický vývoj procedurálního generování ve hrách	11
2.2.1	Beneath Apple Manor	12
2.2.2	Rogue a žánr roguelike	12
2.2.3	Elite	13
2.3	Generování dungeonů ve 2D hrách	14
2.3.1	Požadavky na generování dungeonových map	14
2.4	Přehled metod generování	14
2.4.1	Náhodné místnosti a chodby	15
2.4.2	Binary Space Partitioning	15
2.4.3	Náhodná procházka	16
2.4.4	Buněčné automaty	16
2.4.5	Bludiště	17
2.4.6	Další metody	18
2.5	Shrnutí způsobů generování	18
3	Experimentální část	20
3.1	Použité technologie	20
3.1.1	Herní engine Godot	20
3.2	Návrh systému generování	20
3.2.1	Reprezentace mapy	20
3.2.2	Struktura generátoru	21
3.2.3	Vizualizace mapy a hráče	22
3.2.4	Ovládání generování a zobrazení statistik	23
3.3	Implementace generovacích metod	24
3.3.1	Náhodné místnosti a chodby	24
3.3.2	Binary Space Partitioning	26
3.3.3	Náhodná procházka	27
3.3.4	Buněčné automaty	28
3.3.5	Bludiště	29
3.4	Metodika měření a vyhodnocení	31
4	Výsledky	32
4.1	Výsledky jednotlivých algoritmů	32
4.1.1	Náhodné místnosti a chodby	32
4.1.2	Binary Space Partitioning	33
4.1.3	Náhodná procházka	34

4.1.4	Buněčné automaty	35
4.1.5	Bludiště	36
4.2	Srovnání algoritmů	37
4.2.1	Průměrný čas generování	37
4.2.2	Průměrné pokrytí	38
4.2.3	Škálování s velikostí mapy	39
4.2.4	Kompromis mezi rychlostí a pokrytím	40
4.2.5	Rozptýl výsledků na velké mapě	41
4.3	Souhrnná tabulka výsledků	42
4.4	Shrnutí výsledků	42
5	Diskuse	44
5.1	Interpretace výsledků	44
5.2	Silné a slabé stránky jednotlivých metod	44
5.3	Zkušenosti z implementace	45
5.4	Souvislost s předchozím výzkumem	45
5.5	Možnosti dalšího rozšíření práce	46
	Závěr	47
	Conclusions	48
	A Odkaz na repozitář projektu	49
	B Obsah elektronických dat	50
	Seznam zkratk	51
	Literatura	52

Seznam obrázků

1	Schéma architektury systému generování dungeonových map . . .	22
2	Uživatelské rozhraní aplikace pro generování map	24
3	Ukázka mapy generované pomocí algoritmu náhodných místností a chodeb (45×45)	26
4	Ukázka mapy generované pomocí algoritmu BSP (45×45)	27
5	Ukázka mapy generované pomocí algoritmu Náhodné procházky (45×45)	28
6	Ukázka mapy generované pomocí algoritmu Buněčných automatů (45×45)	29
7	Ukázka mapy generované pomocí algoritmu Bludiště (45×45) . .	30
8	Místnosti a chodby – průměrný čas generování podle velikosti mapy	32
9	Místnosti a chodby – průměrné pokrytí podle velikosti mapy . . .	33
10	BSP – průměrný čas generování podle velikosti mapy	33
11	BSP – průměrné pokrytí podle velikosti mapy	34
12	Náhodná procházka – průměrný čas generování podle velikosti mapy	34
13	Náhodná procházka – průměrné pokrytí podle velikosti mapy . . .	35
14	Buněčné automaty – průměrný čas generování podle velikosti mapy	35
15	Buněčné automaty – průměrné pokrytí podle velikosti mapy . . .	36
16	Bludiště – průměrný čas generování podle velikosti mapy	36
17	Bludiště – průměrné pokrytí podle velikosti mapy	37
18	Srovnání průměrného času generování všech algoritmů	38
19	Srovnání průměrného času generování na mapě 150×150	38
20	Srovnání průměrného pokrytí všech algoritmů	39
21	Škálování času generování s počtem buněk mapy	39
22	Škálování pokrytí s počtem buněk mapy	40
23	Kompromis mezi rychlostí a pokrytím	41
24	Rozptyl času generování na mapě 150×150	41
25	Rozptyl pokrytí na mapě 150×150	42

Seznam tabulek

1	Srovnání vybraných metod generování dungeonových map [1, 2]. Variabilita se vztahuje ke strukturální podobě generovaného prostředí.	19
2	Ukázka reprezentace mapy v mřížce 5×5	21
3	Průměrný čas generování a pokrytí	42

1 Úvod

Procedurální generování obsahu, neboli *Procedural Content Generation* (PCG), představuje v herním vývoji významný přístup, který umožňuje automatizovaně vytvářet části herního světa na základě předem definovaných pravidel a algoritmů. V oblasti digitálních her se tento princip uplatňuje zejména tam, kde je žádoucí vysoká variabilita prostředí, znovuhratelnost nebo snížení nároků na ruční tvorbu obsahu. Jednou z typických oblastí využití je generování dungeonových map, které tvoří důležitou součást 2D her.

Dungeonové mapy kladou na generování specifické požadavky. Nestačí, aby byly pouze náhodné, ale měly by splňovat základní podmínky hratelnosti, jako jsou průchodnost, přiměřené rozložení prostoru a vhodné rozmístění herních prvků. Volba generovací metody proto výrazně ovlivňuje výsledný vzhled mapy i herní zážitek hráče.

Cílem práce je představit vybrané metody procedurálního generování dungeonových map ve 2D hrách, analyzovat jejich vlastnosti a implementovat jejich praktické ukázky v herním engineu Godot. Součástí práce je návrh systému v podobě jednoduché hry, který umožní vybrané algoritmy realizovat, vizualizovat a následně je mezi sebou porovnat. Důraz je kladen jak na teoretické vymezení problematiky, tak na praktickou implementaci a zhodnocení výsledků.

Práce je rozdělena do pěti hlavních kapitol: teoretické základy PCG a popis vybraných algoritmů generování, návrh a implementace praktické části, popis naměřených hodnot, diskuse o metodách a naměřených výsledcích a závěr.

2 Současný stav řešené problematiky

2.1 Procedurální generování obsahu ve hrách

2.1.1 Vymezení a definice pojmu

Procedurální generování ve hrách lze vymezit jako algoritmickou tvorbu herního obsahu s omezeným nebo nepřímým zásahem člověka. Herní obsah zahrnuje široké spektrum prvků, například mapy, úrovně, pravidla, úkoly, zbraně či předměty a jejich vlastnosti. V závislosti na konkrétním návrhu může ovlivňovat nejen prostorovou strukturu hry, ale také její mechaniky, obtížnost či samotný průběh hraní [1].

V současnosti je pojem generování obsahu často spojován s metodami umělé inteligence, tato práce se však zaměřuje výhradně na algoritmické postupy založené na předem definovaných pravidlech.

Pojem hra je v odborné literatuře obtížné jednoznačně vymezit, protože zahrnuje široké spektrum forem lišících se cíli, pravidly, mírou interaktivity i použitou platformou. V kontextu této práce je proto termín hra chápán jako digitální videohra realizovaná v reálném čase, ve které hráč interaguje s virtuálním prostředím přes definované herní mechaniky. Práce se zaměřuje především na 2D videohry určené pro osobní počítače, případně další běžné platformy.

2.1.2 Výhody a nevýhody procedurálního generování

Procedurální generování obsahu přináší řadu výhod. Mezi hlavní patří vysoká variabilita výsledného obsahu a podpora znovuhratelnosti, protože každé spuštění produkuje unikátní prostředí. Dalšími výhodami jsou snížení nároků na ruční tvorbu obsahu a možnost generování rozsáhlých herních světů s minimálními paměťovými nároky, jak demonstruje například hra *Elite* (viz podkapitola 2.2.3). Parametrizací generátoru lze navíc přizpůsobit výsledné prostředí různým herním situacím nebo obtížností [1, 2].

Využití PCG s sebou však nese také několik významných omezení. Jedním z hlavních problémů je nižší míra kontroly nad výsledným obsahem. U ručně navržených úrovní má vývojář plnou kontrolu nad tempem hry, klíčovými momenty, rozmístěním výzev, předmětů či nepřátel. Naproti tomu u procedurálního generování je výsledná podoba světa závislá na implementovaném algoritmu a zvolených parametrech. To může vést k nevyváženým úrovním, nelogickému rozmístění prvků nebo situacím, které jsou příliš jednoduché či naopak extrémně náročné. Vývojář proto musí věnovat značné úsilí návrhu, ladění a testování generátoru, aby bylo dosaženo uspokojivé kvality výsledného obsahu.

Dalším rizikem je možnost vzniku repetitivního nebo uměle působícího obsahu. I přestože je obsah generován automaticky a náhodně, vychází z omezené množiny předem definovaných pravidel. Pokud není systém navržen dostatečně variabilně, může hráč postupně rozpoznat opakující se vzorce, což může negativně ovlivnit herní zážitek. Tento problém je sice možné minimalizovat vhodným

návrhem algoritmu a důkladným laděním, nicméně zcela eliminovat jej bývá obtížné [1].

2.1.3 Přístupy k procedurálnímu generování

PCG lze klasifikovat podle více kritérií, například podle způsobu reakce na hráče nebo podle deterministické či stochastické povahy výstupu.

Nejčastěji se lze v praxi setkat s adaptivním či neadaptivním přístupem, a to vzhledem k chování hráče. Neadaptivní (statické) generování vytváří obsah bez ohledu na chování hráče a jeho průběh hry. Všechny generované struktury vycházejí ze stejných pravidel a reakce hráče na podněty neovlivní výsledek. Naopak adaptivní generování pracuje s informacemi o hráči. Systém může sledovat úspěšnost hráče, rychlost reakcí, délku průchodu úrovně, využívané herní prvky nebo způsob řešení situací a na základě těchto dat dále upravovat generovaný obsah.

Dalším důležitým rozdělením je deterministický a stochastický přístup. Deterministické generování vychází z pevně daných počátečních hodnot, například seed, což je textová nebo číselná hodnota, která je výchozím bodem pro generátor a určuje počáteční podmínky algoritmu, podle kterých je následně obsah generován. Při stejném vstupu vždy produkuje stejný výstup. Tento přístup umožňuje reprodukovatelnost výsledků, což je výhodné například při testování nebo sdílení konkrétních map mezi hráči. Stochastické generování naproti tomu pracuje s náhodností bez pevně stanoveného počátečního stavu, takže výsledky se liší i při opakovaném spuštění algoritmu.

Z hlediska procesu generování lze rozlišit konstruktivní metody a přístup označovaný jako generate-and-test. Konstruktivní metody vytvářejí obsah postupně podle předem definovaných pravidel. Každý krok generování přímo přispívá k vytvoření výsledné struktury. Naproti tomu metoda generate-and-test nejprve vytvoří potenciální řešení, které je následně ověřeno podle stanovených kritérií. Pokud výsledek nevyhovuje, proces se opakuje. Tento přístup může být výpočetně náročnější, ale umožňuje lépe kontrolovat výsledné vlastnosti mapy.

V literatuře se rovněž objevuje pojem smíšené autorství (mixed-authorship), kdy se na tvorbě obsahu podílí jak algoritmus, tak člověk. Systém může například generovat návrhy úrovní, které následně autor upravuje, případně může reagovat na částečně ručně vytvořenou strukturu.

Plně automatická generace probíhá bez zásahu člověka během samotného vytváření obsahu. Algoritmus pracuje pouze na základě vstupních parametrů. Tento způsob je typický zejména pro roguelike hry a další tituly, u nichž je cílem vysoká variabilita jednotlivých průchodů [1].

2.2 Historický vývoj procedurálního generování ve hrách

Počátky procedurálního generování sahají již do 70. let 20. století, kdy byl výpočetní výkon i dostupná operační paměť velmi omezené. Tyto limity výrazně ovlivnily množství ručně vytvářeného obsahu, a vývojáři proto hledali způsoby,

jak vytvářet variabilní herní prostředí s minimálními nároky na operační paměť (RAM). Právě z tohoto důvodu se začaly prosazovat procedurální přístupy [1].

2.2.1 Beneath Apple Manor

Jednou z prvních her, které implementovaly PCG, byla Beneath Apple Manor z roku 1978. Autorem titulu byl Don Worth a hru vydalo studio The Software Factory. Hra byla určena především pro počítač Apple II.

Herní prostředí v Beneath Apple Manor bylo generováno algoritmicky při každém novém spuštění. Rozložení místností, chodeb ani jednotlivých herních prvků či nepřátel nebylo nikdy zcela totožné, a právě tím byla hra přívětivá pro znovuhratelnost. Cílem hry bylo získat zlaté jablko a postup bylo možné ukládat, takže po smrti hráče se načetla uložená úroveň znovu, avšak hráč přišel o část svých atributů.

Autor hry čerpal inspiraci z programu Dragon Maze, který rovněž využíval náhodné výpočty ke generování bludišť. Tento princip však dále rozvinul a navrhl algoritmus vytvářející funkční a hratelnou strukturu dungeonů složenou z místností propojených chodbami. Hra navíc umožňovala hráči ovlivnit některé parametry generování, například počet místností na jednotlivých patrech, barevnou variantu zobrazení nebo samotnou obtížnost. Po vytvoření struktury úrovně algoritmus náhodně rozmístil také nepřátele, poklady a další herní objekty. S rostoucí hloubkou dungeonů se zároveň zvyšovala obtížnost hry, a to zejména síla nepřátel.

Použitý algoritmický přístup lze zařadit mezi rané příklady způsobu generování založeného na místnostech a chodbách, který se později stal jedním ze základních modelů využívaných v hrách typu roguelike. Beneath Apple Manor tak představuje významný historický příklad využití těchto metod, jehož principy byly dále rozvíjeny v následujících dekáдах [3, 4].

2.2.2 Rogue a žánr roguelike

Hra Rogue si oproti výše zmíněnému Beneath Apple Manor získala výrazně větší popularitu a měla zásadní vliv na další vývoj podobných her. Právě podle této hry vznikl podžánr označovaný jako roguelike, z jehož principů dodnes vychází celá řada titulů.

Roguelike hry bývají typické náhodně generovanými úrovněmi, tahovým systémem, permanentní smrtí postavy (permadeath) a zpravidla i vyšší obtížností. Náhodné generování zde neplní pouze technickou funkci, ale představuje zásadní designový prvek, který podporuje průzkum, strategické rozhodování i adaptaci hráče na proměnlivé prostředí.

Hra vyšla v roce 1980 a jejími autory jsou Michael Toy, Glenn Wichman a Ken Arnold. Titul původně vznikl pro unixové systémy a zobrazoval herní svět pomocí ASCII znaků v textovém terminálu. Jednotlivé prvky prostředí byly reprezentovány různými symboly, například hráč symbolem @, stěny znakem #, nepřátelé písmeny abecedy. Tento způsob zobrazení nebyl pouze důsledkem technických

omezení tehdejšího výpočetního výkonu, ale umožňoval poměrně flexibilní práci s herním světem a jeho generováním.

Rogue výrazně zpopularizovalo princip permanentní smrti. Postup hrou nebylo možné ukládat a jakmile hráč zemřel, musel začít zcela od začátku. Hráči tak zůstávaly pouze nabyté zkušenosti a znalost herních mechanik. Smrt zde proto nepředstavovala pouze neúspěch, ale také nedílnou součást učení se hernímu systému.

Cílem celé hry je najít Amulet of Yendor, který se nachází v nejhlubším patře dungeonů, a následně se vrátit zpět na povrch. Dungeon obsahuje několik pater (tradičně 26) a každé z nich je generováno procedurálně. Patra se skládají z místností propojených chodbami a jejich rozložení se při každém novém průchodu mění. Náhodně jsou rozmístěni také nepřátelé, pasti, poklady, zbraně a další předměty.

Prvek náhody se projevuje i ve vlastnostech předmětů. Například lektvary, svitky či prsteny mají při každé nové hře odlišné označení a jejich účinek není hráči předem znám. I pokud hráč nalezne stejný typ předmětu jako v předchozí hře, jeho vlastnosti se mohou lišit. Tím je posílena nejistota a nutnost experimentování.

Kombinace procedurálně generovaných úrovní a permanentní smrti zajišťuje, že každý průchod hrou je unikátní. Hráč se tak nemůže spoléhat na zapamatování konkrétní mapy, ale musí se přizpůsobovat aktuálně vygenerovaným podmínkám. Právě tento princip se stal jedním ze základních stavebních kamenů celého roguelike žánru [5, 6].

2.2.3 Elite

Dalším významným titulem využívajícím procedurální generování je hra Elite z roku 1984, kterou vytvořili David Braben a Ian Bell. Na rozdíl od předchozích her se nejedná o dungeon, ale o vesmírný simulátor obchodování a soubojů. Přesto je Elite z hlediska procedurálního generování velmi důležitým milníkem.

Hra byla původně vydána pro počítač BBC Micro, který disponoval velmi omezenou pamětí. Z tohoto důvodu nebylo možné ukládat rozsáhlý herní svět přímo v paměti. Vývojáři proto zvolili přístup, kdy byl herní svět generován algoritmicky na základě předem definovaného číselného základu. Pomocí tohoto deterministického postupu bylo možné při každém spuštění znovu vytvořit stejnou galaxii bez nutnosti jejího explicitního uložení.

Elite obsahovalo osm galaxií a každá z nich zahrnovala stovky hvězdných systémů. Vlastnosti jednotlivých systémů, jako je jejich název, ekonomika, úroveň technologického rozvoje či typ vlády, byly generovány pomocí pseudonáhodných algoritmů. Procedurálně vznikaly také ceny komodit a další herní parametry.

Tento přístup ukázal, že procedurální generování není omezeno pouze na tvorbu dungeonů nebo jednotlivých úrovní, ale může být využito i pro vytvoření rozsáhlých herních světů. Elite tak demonstrovalo, že algoritmické generování obsahu představuje efektivní řešení zejména v podmínkách s omezeným výpočetním výkonem a omezenými paměťovými zdroji [7].

2.3 Generování dungeonů ve 2D hrách

Dungeonové mapy se typicky skládají z místností a chodeb, obsahují jasně definované průchozí a neprůchozí oblasti a musí splňovat požadavky na hratelnost. Generování takových struktur proto vyžaduje algoritmy, které dokáží zajistit nejen variabilitu výsledku, ale i splnění základních strukturálních vlastností [1, 2].

2.3.1 Požadavky na generování dungeonových map

Při návrhu algoritmu pro generování dungeonové mapy je nutné zohlednit několik základních požadavků, které vycházejí jak z technických omezení, tak z hlediska hratelnosti. Generovaná mapa by neměla být pouze náhodným shlukem místností a chodeb, ale strukturovaným prostředím, které umožňuje smysluplný průchod.

Jedním ze základních požadavků je průchodnost mapy neboli connectivity. Všechny klíčové části mapy by měly být vzájemně dosažitelné, aby nedocházelo k izolovaným oblastem bez možnosti přístupu. Tento požadavek je zásadní zejména u her, kde je cílem projít celou úroveň nebo nalézt konkrétní objekt. Výjimkou jsou hry, ve kterých je neprůchodnost záměrná a hráč se do izolovaných částí dostane například prokopáním.

Dalším důležitým aspektem je kontrola struktury a rozložení prostoru. Mapa by měla obsahovat přiměřený poměr otevřených a uzavřených oblastí, vhodnou velikost místností a dostatečně široké průchody, aby hráč mohl bez problémů projít. Příliš husté nebo naopak příliš prázdné prostředí může negativně ovlivnit hratelnost i orientaci hráče.

Významným požadavkem je rovněž variabilita výsledků. Opakované spuštění generátoru by mělo vést k dostatečně odlišným mapám, aby byl zachován prvek znovuhratelnosti. Zároveň je vhodné, aby generování respektovalo určitá pravidla nebo omezení, která zajistí konzistentní kvalitu výsledku.

Z technického hlediska je důležitá i výpočetní náročnost algoritmu. Generování by mělo probíhat v přiměřeném čase a nemělo by příliš zatěžovat systém, zejména pokud je realizováno za běhu hry (online generování). U jednodušších 2D dungeonových her je možné využít algoritmy s relativně nízkou časovou složitostí, jelikož požadavky na generování jsou značně nižší.

Možnost nastavit velikost mapy, počet místností, hustotu chodeb nebo další vlastnosti generátoru umožňuje lépe přizpůsobit výsledné prostředí konkrétním požadavkům hry a usnadňuje testování různých konfigurací [1, 2].

2.4 Přehled metod generování

Metody generování dungeonových map se liší způsobem konstrukce prostoru, mírou kontroly nad výslednou strukturou, typem generovaného prostředí a výpočetní náročností. Některé algoritmy vytvářejí mapu postupně přidáváním jednotlivých prvků, jiné pracují s rozdělováním prostoru nebo s pravidly aplikovanými na mřížku buněk.

Ve 2D hrách převažují metody založené na náhodném umístění místností a jejich následném propojování, rozdělování velkého prostoru na menší části nebo algoritmy založené na takzvané náhodné procházce. Každý z těchto přístupů má své výhody i omezení a je vhodný pro odlišný typ herního prostředí [1, 2].

2.4.1 Náhodné místnosti a chodby

Generování pomocí náhodného umístění místností a chodeb patří mezi nejpoužívanější a zároveň nejjednodušší přístupy z hlediska implementace.

Na mapě se nejprve generují místnosti různých velikostí, obvykle obdélníkového nebo čtvercového tvaru. Během generování se kontroluje, aby se nově vznikající místnost nepřekrývala s již existujícími. Výsledkem je množina místností reprezentovaných například souřadnicemi a rozměry. Místnosti jsou následně propojeny chodbami. Ty bývají realizovány jako jednoduché horizontální a vertikální průchody spojující středy dvou místností nebo jejich nejbližší body. Častým řešením je propojení ve tvaru písmene L, kdy jsou dvě místnosti spojeny dvojicí kolmých chodeb.

Pro zajištění průchodnosti mapy se využívají algoritmy z oblasti teorie grafů. Jedním z typických řešení je použití minimální kostry grafu (Minimum Spanning Tree), která zaručuje propojení všech místností při minimálním počtu chodeb.

Výhodou této metody je především její jednoduchost. Generované dungeony obvykle obsahují přehledně oddělené místnosti a srozumitelnou síť propojení. Nevýhodou je menší variabilita, protože místnosti mají pravidelný tvar a prostředí tak může působit příliš uniformně. Dalším problémem bývá méně efektivní využití prostoru, kdy mezi jednotlivými místnostmi zůstávají větší nevyužité oblasti. Komplikace může přinášet i samotné propojování. U jednodušších implementací mohou vznikat kolize chodeb, nevhodné křížení nebo příliš dlouhé průchody.

Metoda je často kombinována s dalšími postupy nebo doplněna o dodatečná pravidla, která omezují velikosti a vzájemné vzdálenosti místností, délku chodeb nebo celkové rozložení prostoru [1, 2].

2.4.2 Binary Space Partitioning

Metoda Binary Space Partitioning (BSP) patří mezi přístupy založené na systematickém rozdělování prostoru. Celá mapa se nejprve reprezentuje jako jeden velký obdélníkový nebo čtvercový prostor, tedy kořen stromu, který je postupně rozdělován na menší části. Každé rozdělení může probíhat horizontálně nebo vertikálně a výsledkem je hierarchická struktura podobná binárnímu stromu.

Generování obvykle začíná rozdělením celé mapy na dvě části. Tyto části se dále rekurzivně dělí, dokud nedosáhnou určité minimální velikosti. Každé rozdělení vytváří dva nové uzly ve stromové struktuře. Listové uzly představují konečné oblasti.

Ve vzniklých oblastech jsou vytvořeny místnosti, jejichž velikost je o něco menší než velikost daného listu. Tím vzniká přirozený prostor mezi místnostmi, který může být využit například pro generování chodeb nebo dalších herních

struktur. Místnosti jsou propojovány postupným spojováním sousedních místností stromové struktury.

Zmíněný postup umožňuje relativně snadno zajistit vzájemnou dosažitelnost všech místností. Zásadní výhodou BSP je dobrá kontrola nad velikostí a rozmístěním jednotlivých částí mapy. Parametry dělení prostoru umožňují ovlivnit výsledný vzhled prostředí a předejít situacím, kdy by vznikaly příliš malé nebo naopak příliš velké místnosti.

Nevýhodou této metody je poměrně pravidelný a místy uniformní vzhled generovaného prostředí, protože výsledná struktura často odráží samotný způsob dělení prostoru [1, 2].

2.4.3 Náhodná procházka

Metoda Náhodné procházky (Drunkard's Walk) je založena na jednoduchém principu postupného rozšiřování prostoru pomocí pohybu náhodným směrem. Algoritmus začíná na předem dané počáteční pozici na mapě a v každém kroku se náhodně přesune do jednoho ze sousedních polí. Směr pohybu je obvykle vybírán ze čtyř základních směrů v mřížce, tedy nahoru, dolů, doleva nebo doprava.

Při každém kroku je aktuální buňka označena jako průchozí prostor. Postupným opakováním tohoto procesu vzniká síť chodeb a otevřených oblastí, jejichž tvar závisí na délce a průběhu Náhodné procházky. Výsledná struktura mapy bývá velmi nepravidelná a rozdílná při každém generování.

Proces generování obvykle pokračuje na základě předem stanoveného počtu kroků, nebo dokud není dosaženo určitého poměru průchozích buněk. V některých variantách algoritmu může být použito více nezávislých „chodců“, kteří se po mapě pohybují současně a vytvářejí tak komplexnější strukturu prostředí.

Výhodou této metody je velmi jednoduchá implementace a schopnost generovat přirozeně působící struktury. Nevýhodou je naopak menší kontrola nad výsledným tvarem mapy a obtížnější zajištění rovnoměrného využití prostoru. V některých případech může docházet k tomu, že většina průchozích oblastí vzniká v blízkosti počáteční pozice algoritmu, zatímco jiné části mapy zůstávají téměř nevyužité.

Metoda je vhodná pro generování prostředí připomínajícího jeskyně [1].

2.4.4 Buněčné automaty

Buněčné automaty představují přístup založený na aplikaci specifických pravidel na mřížku buněk. Mapa je v tomto případě reprezentována jako dvourozměrná mřížka, kde každá buňka může být například ve stavu podlaha nebo prázdný prostor. Na začátku generování je mapa inicializována náhodným rozdělením těchto stavů, což vytváří základní strukturu pro další postup.

Celá mapa se postupně prochází a stav jednotlivých buněk se aktualizuje na základě stavu jejich sousedních buněk. Nejčastěji se používá takzvané Mooreovo sousedství, které zahrnuje osm buněk obklopujících aktuální buňku. Pravidla generování bývají různá. Mohou například určovat, že buňka zůstane průchozí

pouze tehdy, pokud má dostatečný počet průchozích sousedů. Buňka zůstane prázdnou, nebo se změní na stěnu, pokud je obklopena větším množstvím prázdného prostoru.

Opakovaným aplikováním těchto pravidel dochází k postupnému vyhlazování vytvořené struktury. Izolované buňky nebo malé skupiny buněk se postupně odstraňují a vznikají větší souvislé oblasti. Výsledkem jsou mapy s nepravidelnými tvary.

Výhodou této metody je schopnost generovat organicky působící prostředí, které se liší od klasických dungeonů. V literatuře je algoritmus popisován jako výpočetně efektivní, protože každý simulační krok prochází mapu v lineárním čase vzhledem k počtu buněk. Nevýhodou může být menší kontrola nad přesnou strukturou mapy a také nutnost dodatečných kroků, které zajistí průchodnost celé mapy. V implementaci může často docházet k vytváření izolovaných částí a v některých případech je proto nutné je zcela odstranit nebo dodatečně propojit, což negativně ovlivňuje složitost algoritmu [1, 2].

2.4.5 Bludiště

Metody generování bludišť vytvářejí struktury tvořené převážně úzkými chodbami, které tvoří souvislou síť průchodů. Mapa je v tomto případě reprezentována jako mřížka buněk oddělených stěnami. Samotné generování spočívá v postupném odstraňování těchto stěn tak, aby vznikly propojené průchody.

Algoritmy často vycházejí z principů grafových algoritmů, například prohledávání do hloubky (Depth-First Search) s návratem (Backtracking) nebo algoritmů typu Prim či Kruskal. V případě algoritmu založeného na prohledávání do hloubky se začíná v náhodně zvolené buňce, ze které se postupně prochází do sousedních dosud nenavštívených buněk. Při každém kroku se odstraní stěna mezi aktuální buňkou a vybraným sousedem. Pokud již neexistuje žádný nenavštívený soused, algoritmus se vrací zpět k předchozí buňce.

Primův algoritmus vytváří bludiště postupným rozšiřováním již vytvořené oblasti. Algoritmus začíná v náhodné buňce a postupně vybírá sousední stěny, které oddělují navštívené buňky od nenavštívených. Pokud odstranění vybrané stěny spojí dvě dosud nepropojené oblasti, je tato stěna odstraněna a nová buňka se stane součástí bludiště.

Kruskalův algoritmus naproti tomu pracuje s množinou všech stěn v mapě. Stěny jsou postupně vybírány v náhodném pořadí a odstraňují se pouze tehdy, pokud jejich odstranění nespojí dvě buňky, které již patří do stejné části bludiště. Tím se postupně vytváří souvislá síť průchodů.

Výsledkem je takzvané perfektní bludiště, ve kterém mezi dvěma body existuje právě jedna cesta. Taková struktura neobsahuje cykly a vytváří spleť sítí chodeb. Tento typ generování je vhodný zejména pro hry zaměřené na průzkum a orientaci v prostoru.

Nevýhodou této metody je omezený výskyt větších otevřených oblastí, protože generovaná struktura je tvořena převážně úzkými chodbami. Z tohoto důvodu je implementace často kombinována s jinými metodami, které umožňují

vytvářet větší místnosti. V některých případech se po vygenerování bludiště odstraní vybrané stěny, čímž vznikají cykly a otevřenější struktura mapy [1, 2].

2.4.6 Další metody

Kromě výše uvedených přístupů existuje celá řada dalších metod generování dungeonových map. Patří mezi ně například metody založené na grafech, kde je struktura mapy reprezentována jako graf. Jednotlivé vrcholy grafu představují místnosti a hrany určují jejich vzájemné propojení. Na základě takto vytvořené struktury je vytvořena konkrétní geometrická podoba mapy.

Další možností je využití procedurálních gramatik, které definují pravidla pro vytváření jednotlivých částí mapy. Pravidla se postupně aplikují a vytvářejí požadovanou strukturu, čímž lze poměrně přesně řídit strukturu generovaného prostředí. Metoda je vhodná především pro hry, které vyžadují specifické uspořádání jednotlivých herních prvků.

V literatuře se vyskytují i metody založené na evolučních algoritmech nebo optimalizačních metodách, které se snaží generované mapy postupně vylepšovat podle specifických kritérií. Obvykle se pracuje s množinou map, která je upravována pomocí genetických operací, jako je mutace nebo křížení.

Modernější přístupy zahrnují metody založené na splňování různých omezení. Jedná se o takzvanou constraint-based generation nebo algoritmy typu Wave Function Collapse, které generují mapy postupným výběrem kompatibilních prvků z předem definované množiny vzorů [8].

Uvedené přístupy jsou často náročnější na implementaci a výpočetní výkon než základní algoritmy popsané v předchozích podkapitolách, a proto se v praxi využívají spíše při offline generování obsahu nebo při návrhu komplexnějších herních světů [1, 2].

2.5 Shrnutí způsobů generování

Jednotlivé přístupy se liší především způsobem konstrukce prostoru, mírou kontroly nad výslednou strukturou a typem generovaného prostředí. Zatímco některé metody vytvářejí spíše pravidelné struktury, jiné generují nepravidelná a organicky působící prostředí.

Základní charakteristiky a využití popsaných metod, které jsou v práci implementovány, shrnuje tabulka 1.

Metoda	Kontrola struktury	Variabilita	Typ prostředí
Místnosti a chodby	vysoká	střední	klasické dungeons
BSP	vysoká	střední	strukturované mapy
Buněčné automaty	nízká	vysoká	jeskyně
Náhodná procházka	nízká	vysoká	organické struktury
Bludiště	střední	střední	labyrinty

Tabulka 1: Srovnání vybraných metod generování dungeonových map [1, 2]. Variabilita se vztahuje ke strukturální podobě generovaného prostředí.

3 Experimentální část

Na základě provedené rešerše bylo pro praktickou část práce vybráno pět algoritmů: generování pomocí Místností a chodeb, Binary Space Partitioning, Buňčné automaty, metoda Náhodné procházky a metoda generování Bludiště.

3.1 Použité technologie

3.1.1 Herní engine Godot

Pro vizualizaci generovaných map byl zvolen herní engine Godot. Jedná se o volně dostupný (open-source) program určený pro vývoj 2D i 3D her. Engine poskytuje integrované nástroje pro práci s grafikou, fyzikou, scénami i skriptováním.

Významnou vlastností Godotu je podpora dlaždicových map (TileMap), které jsou při vývoji 2D her často využívány a výrazně usnadňují samotné grafické ztvárnění. Tento způsob reprezentace je zároveň vhodný i pro implementaci algoritmů procedurálního generování, protože poskytuje jednoduchou manipulaci s jednotlivými buňkami mapy. Engine zároveň umožňuje spouštění a testování ztvárněných metod.

Pro implementaci algoritmů byl použit integrovaný skriptovací jazyk GDScript, který je syntakticky podobný jazyku Python. GDScript zajišťuje přímý přístup k objektům herní scény, komponentám enginu i datovým strukturám, což umožňuje efektivní ladění a testování generovacích metod přímo v prostředí [9, 10].

3.2 Návrh systému generování

3.2.1 Reprezentace mapy

Všechny implementované metody generování pracují nad společnou datovou reprezentací mapy ve formě dvourozměrného pole. Mapa je uložena jako pole řádků, kde každý prvek odpovídá jedné buňce dungeonové mapy. Jednotlivé buňky obsahují celočíselnou hodnotu určující typ dlaždice.

V implementaci jsou rozlišeny tři základní stavy buněk, a to prázdný prostor (0), podlaha (1) a stěna (2). Hodnoty slouží jako základ pro vykreslení odpovídajících dlaždic a následně celé mapy. Hodnota 0 se během generování používá především jako výchozí stav mapy, zatímco hodnoty 1 a 2 představují výslednou strukturu prostředí.

Příklad reprezentace mapy je uveden v tabulce 2. Ukázka ilustruje průchozí oblast (1), která je obklopena stěnami (2), zbytek mapy tvoří nevyužitý prostor reprezentovaný hodnotou 0.

Velikost mapy je určena dvěma parametry, šířkou a výškou, které jsou předávány generovacím algoritmům při jejich spuštění. Jednotlivé metody následně vytvářejí a upravují strukturu mapy podle svého principu generování.

0	0	0	0	0
0	2	2	2	0
2	2	1	2	2
2	1	1	1	2
2	2	2	2	2

Tabulka 2: Ukázka reprezentace mapy v mřížce 5×5

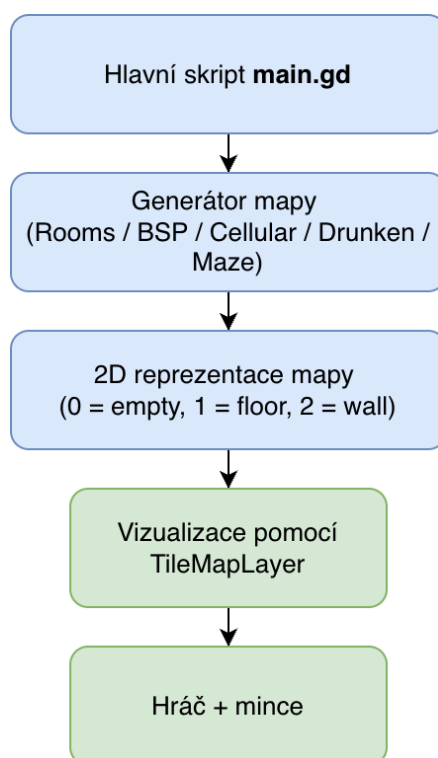
Zvolený způsob reprezentace sjednocuje všechny implementované algoritmy, protože každá metoda vrací výslednou mapu ve stejném formátu. Přístup k jednotlivým buňkám je realizován pomocí jejich souřadnic.

3.2.2 Struktura generátoru

Systém byl navržen modulárně tak, aby bylo možné jednotlivé algoritmy snadno rozšiřovat a vzájemně porovnávat. Každá metoda je implementována v samostatném skriptu a využívá funkci `generate(width, height, seed_value)`, která pro zadané rozměry vrací datovou reprezentaci mapy.

Hlavní řídicí skript `main.gd` zajišťuje propojení uživatelského rozhraní s generovacími algoritmy. Na začátku skriptu jsou definovány konstanty pro typy dlaždic (`EMPTY`, `FLOOR`, `WALL`), výchozí rozměry mapy (60×60), `seed` (1), počet testovacích běhů (200) a pozice grafických prvků v atlasu dlaždic. Při spuštění scény skript naplní rozbalovací nabídku názvy dostupných algoritmů, na základě volby uživatele zavolá příslušný generátor, převezme výslednou mapu a předá ji vizualizační vrstvě. Konkrétní průběh generování a měření je popsán v podkapitole věnované ovládání aplikace.

Architektura navrženého systému je schematicky znázorněna na obrázku 1.



Obrázek 1: Schéma architektury systému generování dungeonových map

3.2.3 Vizualizace mapy a hráče

Pro vizualizaci generovaných map byly v aplikaci použity grafické prvky, takzvané *assets*. Tyto *assets* slouží především k přehlednému zobrazení struktury generované mapy a usnadňují orientaci uživatele. Vizualizace zahrnuje grafiku podlahy, stěn, postavy hráče a sbíratelných objektů.

Zobrazení mapy je realizováno pomocí komponenty `TileMapLayer`. Po dokončení generování je datová reprezentace mapy předána funkci `_draw_map_animated()`, která ji postupně převádí na odpovídající dlaždice, řádek po řádku s krátkou prodlevou mezi jednotlivými řádky. Jednotlivé typy buněk jsou mapovány na konkrétní grafické prvky, podlahu nebo stěnu. Tento způsob postupného vykreslování není nezbytný z hlediska generování, ale výsledná vizualizace působí přívětivěji.

Součástí prototypu je také jednoduchá reprezentace hráče, která slouží především k ověření průchodnosti mapy. Umístování hráče zajišťuje funkce `_place_player_on_floor()`, která pomocí prohledávání do šířky (BFS) nalezne největší souvislou oblast průchozích dlaždic. V této oblasti hledá pozici s volným okolím 3×3 dlaždic, aby hráč nebyl ihned po vytvoření mapy zaseknut ve stěně. Pokud taková pozice neexistuje, je jako záložní řešení použita náhodná dlaždice z největší oblasti o velikosti 2×2 .

S hráčem se lze pohybovat pomocí kláves W, A, S, D. Uživatel může pohybovat kamerou stisknutím a držením pravého tlačítka myši aniž by ovlivnil pozici

hráče. Pro návrat kamery se zameřením na hráče slouží klávesa R, případně klávesy pro pohyb.

Pro doplnění testovacího prostředí byly do mapy přidány sbíratelné objekty ve formě mincí. Funkce `_place_coins_on_floor()` náhodně rozmístí 5 až 50 mincí na podlahové dlaždice v největší souvislé oblasti mapy. Pokud hráč posbírá všechny mince, automaticky se vygeneruje nová mapa se stejnými parametry, jako v případě předchozího generování. Tyto prvky neslouží jako hlavní součást PCG, jedná se pouze o grafické doplnění.

V implementaci je využit volně dostupný balíček 2D Pixel Dungeon Asset Pack, jehož autorem je grafik vystupující pod jménem Pixel_Poem. Tento balíček obsahuje základní sadu grafických prvků určených právě pro tvorbu dungeonových 2D her [11]. Použité assety mají podobu pixelové grafiky v top-down perspektivě, tedy seshora dolů, a svým návrhem přímo odpovídají práci s dlaždicovou mapou, kterou engine využívá. Balíček je volně dostupný a lze jej využít v nekomerčních i komerčních projektech.

Grafická vrstva – hráč a mince – slouží pouze jako podpůrný nástroj pro vizualizaci výsledků generování.

3.2.4 Ovládání generování a zobrazení statistik

Uživatelské rozhraní (UI) umožňuje výběr konkrétní generovací metody a nastavení základních parametrů mapy – šířky, výšky, seedu a počtu opakování měřícího režimu. Pokud chce uživatel využít seed, je nutné zaškrtnout příslušné políčko (checkbox), hodnota seedu je následně předána generátoru a výsledek je plně reprodukovatelný.

Rozhraní obsahuje dvě hlavní tlačítka, *Generate* a *Benchmark*. Po stisknutí tlačítka *Generate* skript, na základě uživatelské volby, vybere příslušný generátor pomocí konstrukce `match` a zavolá jeho funkci

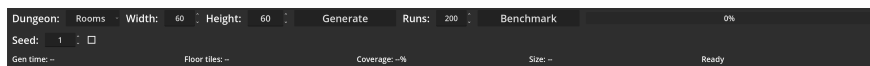
`generate(width, height, seed_value)`, kde `seed_value` je zadaná hodnota seedu, nebo 0, pokud seed není aktivní. Doba generování je dána pomocí funkce

`Time.get_ticks_usec()` a měří výhradně volání generovací funkce, což zajišťuje, že čas není ovlivněn samotným vykreslováním. Ihned po dokončení generování skript spočítá počet průchozích dlaždic a procentuální pokrytí mapy. Údaje se spolu s časem generování zobrazí v UI. Výsledná mapa je poté vizualizována postupným vykreslováním jednotlivých řádků.

Tlačítko *Benchmark* slouží k automatizovanému testování všech implementovaných metod. Proces zajišťuje funkce `run_benchmark()`, která postupně spustí všech pět algoritmů pro každou z pěti definovaných velikostí map, přičemž každou konfiguraci opakuje konfigurovatelným počtem opakování (výchozí hodnota 200). Celkový počet generování v rámci jednoho spuštění benchmarku je dán vzorcem $5 \times 5 \times \text{benchmark_runs}$, kde první hodnota odpovídá počtu algoritmů, druhá počtu velikostí map a třetí je počet opakování. Průběh benchmarku je znázorněn pomocí ukazatele průběhu (progress bar), který průběžně zobrazuje procentuální dokončenost generování. Při tomto režimu není mapa vi-

zualizována ani nejsou umísťovány herní objekty. Jakmile je proces dokončen, uživatel je informován změnou popisku v UI.

Výsledky jednotlivých běhů (algoritmus, rozměry, čas generování, pokrytí, počet podlahových dlaždic) jsou po každém běhu ukládány do souboru `results.csv` pomocí funkce `_log_results()`, jejíž struktura je podrobněji popsána v podkapitole věnované metodice měření.



Obrázek 2: Uživatelské rozhraní aplikace pro generování map

3.3 Implementace generovacích metod

Každý algoritmus z kapitoly 2.4 je implementován jako samostatný skript v adresáři `res://dungeons/algorithms`.

Generátory sdílejí několik společných implementačních prvků. Každý skript rozšiřuje třídu `RefCounted`, nemá tedy vazbu na scénu, a definuje trojici celočíselných konstant pro typy dlaždic: `TILE_EMPTY = 0` (prázdný prostor), `TILE_FLOOR = 1` (podlaha) a `TILE_WALL = 2` (stěna). Vstupním bodem je vždy statická funkce `generate(width, height, seed_value)`, která vrací dvourozměrné pole představující výslednou mapu.

Na začátku funkce `generate()` každý generátor vytvoří lokální instanci `RandomNumberGenerator`. Pokud je předaný parametr `seed_value` větší než 0, je seed nastaven voláním `rng.seed = seed_value`, čímž je zajištěna plná reprodukovatelnost výsledků. V opačném případě je zavolána metoda `rng.randomize()`, která zajistí unikátní průběh náhodných operací. Následně je inicializována prázdná mapa o rozměrech `width × height`, kde jsou všechny buňky nastaveny na hodnotu `TILE_EMPTY`. Výjimkou je generátor Buněčných automatů, který inicializaci mapy a náhodnou počáteční výplň provádí v jednom kroku.

Závěrečnou fází každého generátoru je doplnění stěn pomocí funkce `_add_walls()`, jejíž implementace je ve všech skriptech totožná. Funkce prochází celou mapu a pro každou dlaždici podlahy kontroluje osm okolních buněk (Mooreovo sousedství). Pokud se v sousedství nachází buňka `TILE_EMPTY`, je změněna na `TILE_WALL`. Stěny jsou tedy odvozeny dodatečně z již vytvořené podlahy, nikoli určovány průběžně v každém kroku generování. Kontrola hranic mapy zajišťuje, že se při zápisu stěn nepřistupuje mimo rozsah pole.

3.3.1 Náhodné místnosti a chodby

Implementace algoritmu se nachází v souboru `rooms_generator.gd`.

Počet generovaných místností je odvozen od celkové plochy mapy podle vzorce `room_count = max(5, width * height / AREA_PER_ROOM)`.

Hodnota konstanty `AREA_PER_ROOM = 360` vychází z odhadu průměrné plochy, kterou jedna místnost včetně svého okolí v mapě zabírá. Šířka i výška místnosti jsou generovány náhodně v rozmezí 6 až 16 dlaždic, průměrná velikost místnosti tedy činí přibližně $11 \times 11 = 121$ dlaždic.

Kromě samotné plochy místnosti je však nutné počítat také s prostorem, který zabírají chodby, stěny a povinné jednopolíčkové mezery mezi místnostmi. Na základě empirického pozorování je tento faktor přibližně trojnásobkem plochy místnosti, tedy $121 \times 3 \approx 360$ dlaždic na jednu místnost. Výraz `max(5, ...)` určuje cílovou hodnotu, o kterou se algoritmus pokouší. Na malých mapách bývá výsledný počet místností zpravidla nižší, protože další pokusy často selžou kvůli překryvu nebo nedostatku volného prostoru, zatímco na větších mapách počet úspěšně umístěných místností obvykle roste s plochou. Maximální počet pokusů o umístění místnosti je stanoven jako `room_count * 10`, což zajišťuje dostatečný prostor pro opakované pokusy v případě kolizí s již umístěnými místnostmi, a zároveň zabraňuje nekonečnému cyklování na mapách.

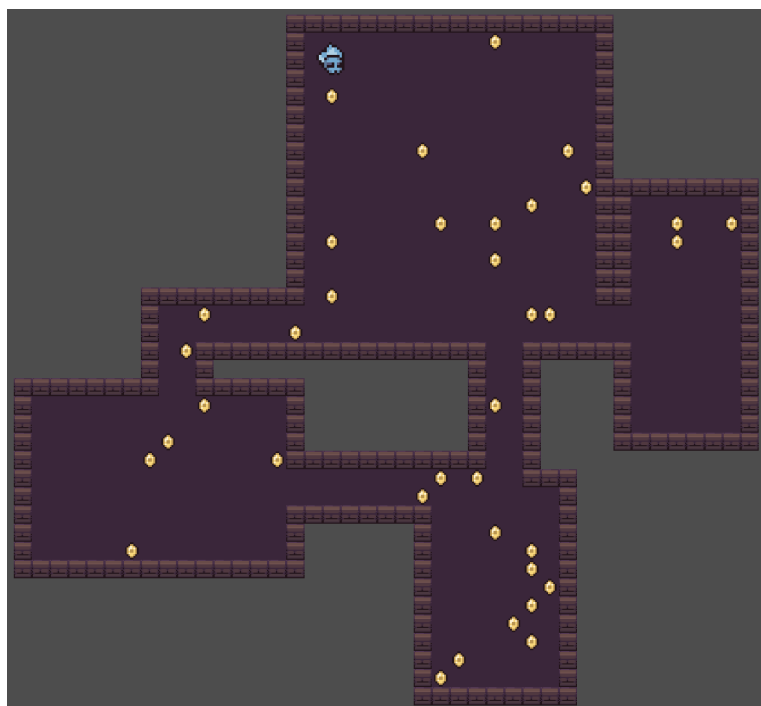
Každá nová místnost je reprezentována obdélníkem, případně čtvercem, typu `Rect2` s náhodně vygenerovanými rozměry. Pozice levého horního rohu místnosti je následně určena tak, aby místnost zůstala uvnitř hranic mapy a zároveň nebyla umístěna přímo na jejím okraji.

Před přidáním nové místnosti do mapy je provedena kontrola překryvu se všemi dříve umístěnými místnostmi. Tato kontrola je realizována pomocí metody `grow(1).intersects(other)`, což znamená, že se testuje nejen skutečný průnik obdélníků, ale i jejich rozšíření o jednu dlaždici na každou stranu. V praxi tak mezi dvěma místnostmi vždy zůstává alespoň jednopolíčková mezera. Pokud by se nová místnost překrývala s některou z již existujících, je zahozena a algoritmus pokračuje dalším pokusem.

V případě, že nová místnost překryv nevykazuje, je přidána do seznamu místností a její vnitřní plocha je zapsána do mapy jako podlaha. To je realizováno dvojicí vnořených cyklů, které procházejí všechny buňky uvnitř obdélníku místnosti a nastavují jejich hodnotu na `TILE_FLOOR`. Tím dojde k vyznačení průchozí oblasti odpovídající nové místnosti.

Každá nová místnost je následně propojena chodbou s předchozí místností v pořadí umístění. První místnost je z tohoto propojování vyloučena, protože v okamžiku jejího přidání ještě neexistuje žádná jiná místnost. Pro propojení se využívají středy obou místností získané pomocí metody `get_center()`. Vlastní vytvoření chodby zajišťuje funkce `_carve_corridor()`, která mezi dvěma zadanými body vyznačí chodbu tvaru písmene L. Algoritmus náhodně rozhoduje, zda bude chodba nejprve vedena ve vodorovném a poté ve svislém směru, nebo opačně.

Chodba není tvořena pouze jednou dlaždici, ale má nastavitelnou šířku. V implementaci je výchozí hodnota parametru `corridor_width` nastavena na 2. Chodba je tedy široká 2 dlaždice a při zápisu dlaždic chodby je současně kontrolováno, zda se zapisované souřadnice nacházejí uvnitř hranic mapy, a to pomocí pomocné funkce `_in_bounds()`.



Obrázek 3: Ukázka mapy generované pomocí algoritmu náhodných místností a chodeb (45×45)

3.3.2 Binary Space Partitioning

Implementace metody BSP se nachází v souboru `bsp_generator.gd`.

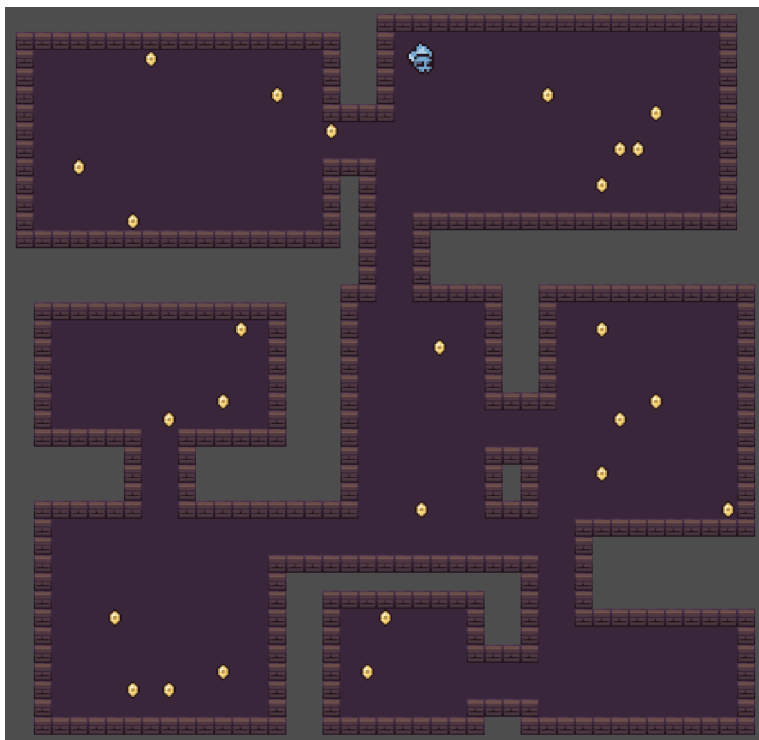
Stromová struktura mapy je reprezentována třídou `Branch`. Každý uzel uchovává svou pozici (`Vector2i`), velikost (`Vector2i`) a náhodné odsazení `padding` typu `Vector4i`, jehož složky udávají vzdálenost místnosti od okraje oblasti, náhodně 2 až 3 dlaždice na každé straně. Díky tomuto odsazení nevznikají místnosti těsně u hranic oblastí a mezi nimi zůstává dostatečný prostor.

Kořenový uzel pokrývá celou mapu zmenšenou o jednu dlaždici na každé straně, tedy oblast od pozice (1,1) o velikosti $(width - 2) \times (height - 2)$. Tím je na okraji mapy ponechán jednopolíčkový rámeček.

Rekurzivní dělení prostoru provádí funkce `split(min_size, rng)`. Směr řezu je určen deterministicky. Pokud je výška oblasti větší nebo rovna šířce (`size.y >= size.x`), dělí se horizontálně, jinak vertikálně. Dělení je ukončeno, pokud by výsledné podoblasti byly menší než `min_size × 2`, minimální velikost je konstanta `MIN_SIZE = 13`. Pozice řezu je volena náhodně v rozmezí 35–65 % délky dělené části, takže vzniklé listy nemusí být stejně velké. V každém vzniklém listu je vykreslena místnost odpovídající ploše uzlu zmenšené o jeho odsazení `padding`.

Po dokončení celého stromu jsou chodby získány metodou `get_corridor_pairs()`. Ta prochází strom rekurzivně a pro každý vnitřní uzel vybere první listový uzel z levého a pravého podstromu a uloží dvojici jejich

středů. Středů místností jsou vypočítány metodou `get_room_center()`, která zohledňuje odsazení `padding`, takže výsledný bod leží vždy uvnitř skutečné průchozí plochy místnosti. Každá chodba má tvar písmene L, nejprve vede ve vodorovném, a poté ve svislém směru. Chodba je široká 2 dlaždice.



Obrázek 4: Ukázka mapy generované pomocí algoritmu BSP (45×45)

3.3.3 Náhodná procházka

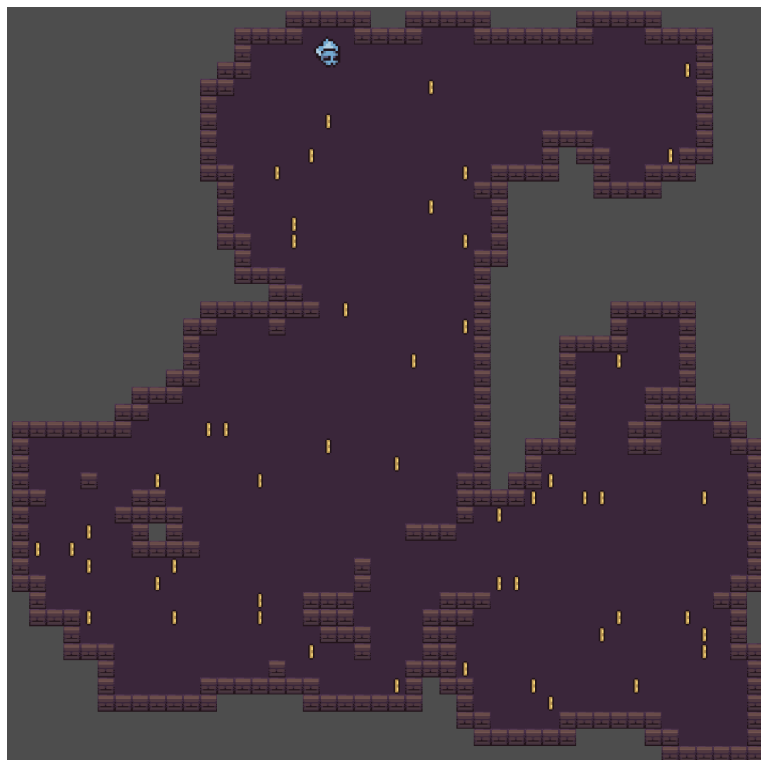
Metoda se nachází v souboru `drunkards_generator.gd`.

Algoritmus začíná ve středu mapy na pozici $\text{width}/2, \text{height}/2$. Kolem počáteční pozice je ihned vytvořena oblast podlahy o velikosti 3×3 dlaždic pomocí funkce `_carve_area()`. Cílový počet průchozích dlaždic je stanoven na 50% celkové plochy mapy podle vzorce (1), kde konstanta `TARGET_COVERAGE` má hodnotu 50.

$$\text{cílový_počet} = \frac{\text{width} \times \text{height} \times \text{TARGET_COVERAGE}}{100} \quad (1)$$

Samotné generování probíhá v cyklu, který pokračuje, dokud počet podlahových dlaždic nedosáhne cílového počtu. V každém kroku je náhodně zvolen jeden ze čtyř směrů (nahoru, dolů, doleva, doprava) a pozice generátoru se posune o jednu dlaždici. Pozice je následně omezena funkcí `clamp()` tak, aby zůstávala alespoň 2 dlaždice od okraje mapy. Tato rezerva zajišťuje, že i při vykreslování oblastí 3×3 nedojde k překročení hranic pole.

Algoritmus rozlišuje dva typy vykreslování. Každých 15 až 25 kroků (interval je volen náhodně) je na aktuální pozici vytvořena oblast 3×3 dlaždic. V ostatních krocích je vykreslena pouze oblast 2×2 dlaždic. Obě varianty jsou realizovány pomocnou funkcí `_carve_area()`, které se předá požadovaná velikost oblasti (3 nebo 2). Funkce zapisuje podlahu pouze tam, kde dosud podlaha nebyla, a vrací počet nově vytvořených dlaždic, aby mohl být průběžně aktualizován celkový počet. Průchodnost celé mapy je zaručena tím, že veškerý prostor vzniká z jedné souvislé procházky.



Obrázek 5: Ukázka mapy generované pomocí algoritmu Náhodné procházky (45×45)

3.3.4 Buněčné automaty

Implementace Buněčných automatů je v souboru `cellular_generator.gd`.

V první fázi je každá vnitřní buňka mapy náhodně nastavena na podlahu s pravděpodobností `FILL_PROBABILITY = 0.45` (45%). Buňky na okraji mapy (první a poslední řádek a sloupec) zůstávají vždy prázdné, čímž vznikne pevný rámec, který zabraňuje rozšíření až k hranicím pole.

Algoritmus provede celkem 5 simulačních kroků, které určuje konstanta `SIMULATION_STEPS = 5`. V každém kroku je vytvořena nová mapa, aby se průběžné změny nepromítaly do výpočtu sousedů v témže kroku. Pro každou buňku funkce `_count_floor_neighbours()` spočítá počet podlahových sousedů v Mooreově sousedství (osm přilehlých buněk). Pokud je tento počet ale-

spoň 4, buňka se stane podlahou, v opačném případě zůstane prázdná. Okrajové buňky ve všech krocích zůstávají prázdné.

Opakováním tohoto pravidla se náhodný šum postupně vyhladí do organických tvarů připomínajících jeskynní systémy. Volba počáteční pravděpodobnosti 45 % a prahu 4 sousedů představuje kompromis mezi dostatečně otevřenými prostory a přirozeně působícími stěnami. Při vyšší hodnotě by vznikaly příliš velké otevřené části, při nižší by byly oblasti méně průchozí, protože by velká část buněk tvořila stěny.

U map generovaných touto metodou se mohou vyskytovat izolované oblasti, protože algoritmus nezaručuje souvislost prostoru. Z tohoto důvodu bylo při umísťování hráče nutné zohlednit průchozí plochu a upřednostnit umístění do největší spojitě oblasti mapy, což bylo nakonec zahrnuto do všech metod.



Obrázek 6: Ukázka mapy generované pomocí algoritmu Buněčných automatů (45×45)

3.3.5 Bludiště

Metoda Bludiště je implementována v souboru `maze_generator.gd`.

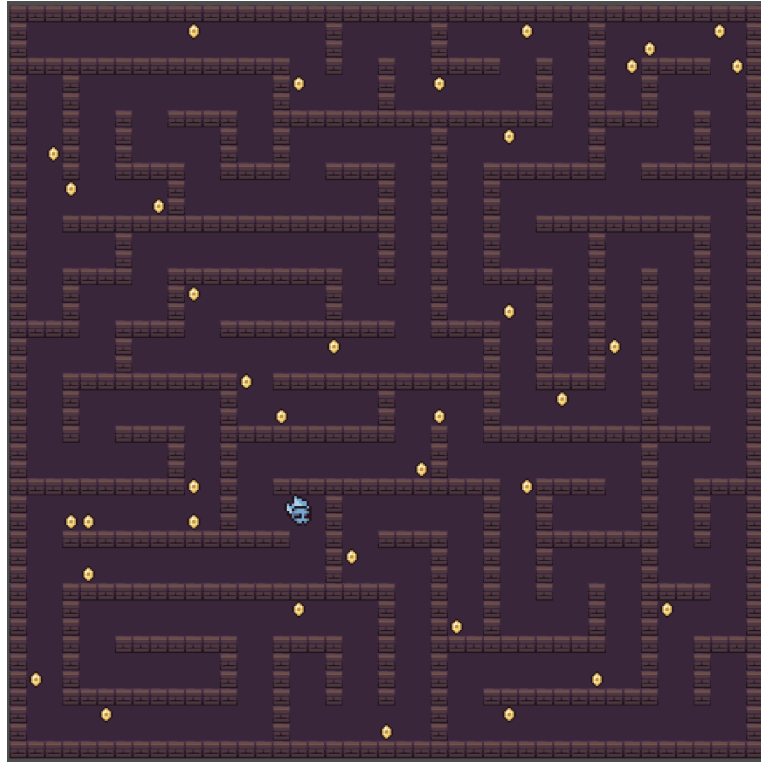
Implementace Bludiště vychází z algoritmu *recursive backtracker* (prohledávání do hloubky se zásobníkem). Mapa je nejprve rozdělena na mřížku abstraktních buněk. Každá buňka zabírá 2×2 dlaždic podlahy a mezi sousedními buňkami je jednodlaždicová mezera, takže na jednu buňku připadá úsek 3×3 dlaždic. Počet buněk závisí na rozměrech mapy, kde odečtení 2 zohledňuje jednodlaždicový

okraj mapy na obou stranách:

$$\text{cells_w} = \max\left(1, \left\lfloor \frac{\text{width} - 2}{3} \right\rfloor\right), \quad \text{cells_h} = \max\left(1, \left\lfloor \frac{\text{height} - 2}{3} \right\rfloor\right). \quad (2)$$

Stav každé buňky je zakódován jako bitová maska čtyř stěn ($N = 1, E = 2, S = 4, W = 8$), na začátku jsou všechny stěny přítomné. Algoritmus startuje v buňce $(0, 0)$ a v každém kroku náhodně vybere nenavštíveného souseda. Ze společné stěny mezi aktuální buňkou a vybraným sousedem je odstraněn příslušný bit na obou stranách, například při pohybu na sever se z aktuální buňky odstraní bit N a ze sousední buňky se odstraní bit S . Pokud aktuální buňka nemá žádného nenavštíveného souseda, algoritmus se vrací zpět po zásobníku. Průchod končí, jakmile jsou navštíveny všechny buňky.

Výsledkem je takzvané perfektní bludiště, ve kterém mezi libovolnými dvěma buňkami existuje právě jedna cesta. Po dokončení prohledávání je buněčná reprezentace převedena na dlaždicovou mapu. Každá buňka je vykreslena jako oblast 2×2 podlahových dlaždic na pozici $(1 + cx \cdot 3, 1 + cy \cdot 3)$. Tam, kde byla stěna odstraněna, je jednodlaždicová mezera mezi sousedními buňkami rovněž vyplněna podlahou o šířce 2 dlaždic, čímž vznikne průchozí koridor.



Obrázek 7: Ukázka mapy generované pomocí algoritmu Bludiště (45×45)

3.4 Metodika měření a vyhodnocení

Pro objektivní porovnání implementovaných algoritmů byl navržen automatizovaný benchmarkový režim, který je součástí aplikace. Jeho spuštění zajišťuje tlačítko *Benchmark* v uživatelském rozhraní. Po aktivaci jsou postupně spuštěny všechny implementované generovací metody pro pět předem definovaných velikostí map: 20×50 , 70×30 , 60×60 , 100×100 a 150×150 dlaždic. Tyto velikosti byly zvoleny tak, aby pokrývaly jak malé, tak velké rozměry map a zároveň umožňovaly posoudit vliv tvaru mapy (čtverec versus obdélník) na chování algoritmů.

Pro každou kombinaci algoritmu a velikosti mapy je generování provedeno opakovaně v 2000 bězích se seedem 05102024. Celkem bylo pořízeno 50 000 záznamů. Díky pevnému seedu jsou výsledky plně reprodukovatelné a pokrytí mapy je pro každou kombinaci algoritmu, velikosti mapy a seedu vždy totožné.

Pomocí funkce `Time.get_ticks_usec()` bylo zajištěno měření času s mikrosekundovou přesností. Měření je výhradně čas funkce `generate(width, height, seed_value)` příslušného algoritmu. Výsledný čas je převeden na milisekundy.

Pro každý běh jsou zaznamenány následující metriky:

- **čas generování** (`gen_time_ms`) – doba trvání samotného algoritmu v milisekundách
- **počet podlahových dlaždic** (`floor_tiles`) – celkový počet průchozích buněk ve výsledné mapě
- **pokrytí** (`coverage`) – podíl průchozích dlaždic z celkového počtu buněk mapy, vyjádřený v procentech:

$$\text{coverage} = \frac{\text{floor_tiles}}{\text{width} \times \text{height}} \times 100 \quad (3)$$

Naměřené hodnoty jednotlivých běhů jsou průběžně ukládány do souboru `results.csv`. Každý záznam obsahuje název algoritmu, rozměry mapy, pořadí běhu, čas generování, pokrytí a počet podlahových dlaždic.

Pro vyhodnocení dat byl vytvořen skript `result_analyser.py` v jazyce Python s využitím knihoven `pandas` a `matplotlib`. Skript načte soubor s výsledky `results.csv` a pro každou kombinaci algoritmu a velikosti mapy vypočítá statistiky: průměr, směrodatnou odchylku, minimum a maximum času generování i pokrytí. Statistiky jsou uloženy do souboru `results_summary.csv`. Následně skript automaticky vygeneruje sadu grafů: sloupcové grafy průměrného času a pokrytí pro jednotlivé algoritmy, škálovací křivky v závislosti na velikosti mapy, boxploty rozložení hodnot a bodový graf kompromisu mezi rychlostí a pokrytím.

4 Výsledky

Všechny grafy v této kapitole byly vygenerovány z dat naměřených benchmarkovým režimem popsaným v předchozí podkapitole. Nadpisy os a legendy grafů jsou uvedeny v anglickém jazyce. Názvy algoritmů v grafech odpovídají anglickým označením *Rooms* (Místnosti), *BSP*, *Drunkard's* (Náhodná procházka), *Cellular* (Buněčné automaty) a *Maze* (Bludiště). V popiscích obrázků je uveden český překlad nadpisu grafu.

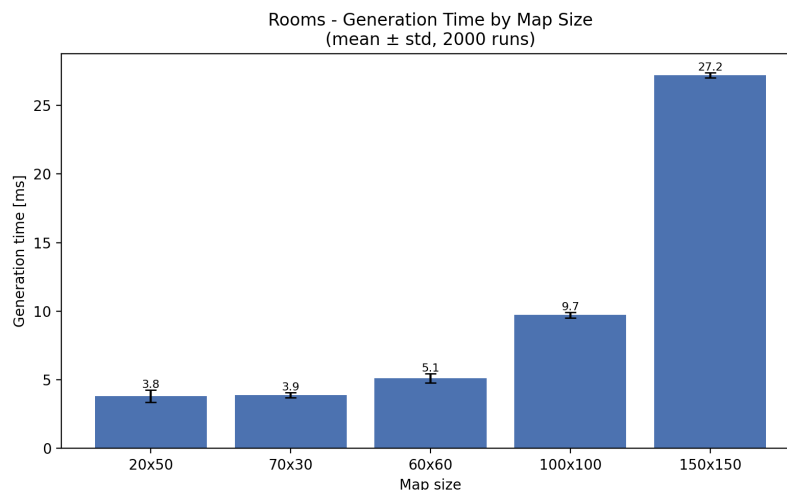
4.1 Výsledky jednotlivých algoritmů

4.1.1 Náhodné místnosti a chodby

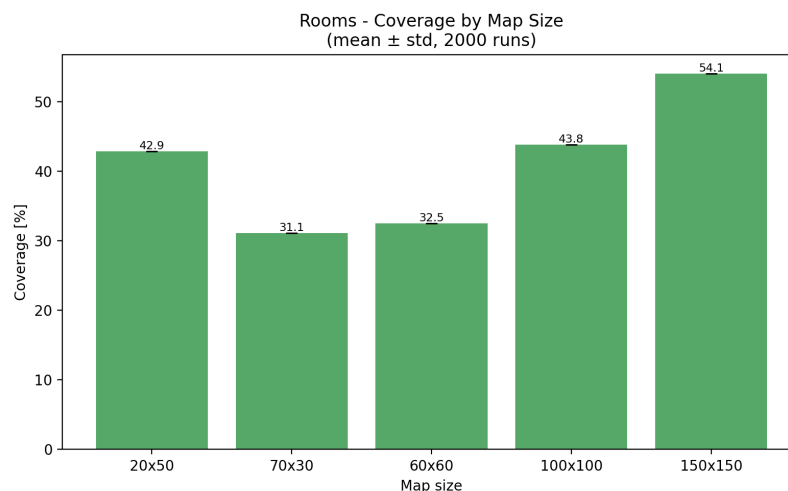
Algoritmus náhodných místností a chodeb vykazuje nízké časy generování na menších mapách. Na mapě 20×50 přibližně 3,8 ms. S rostoucí velikostí mapy čas narůstá výrazněji. Na mapě 150×150 dosahuje průměrný čas přibližně 27,2 ms.

Pokrytí mapy průchozím prostorem má u tohoto algoritmu celkově rostoucí trend s velikostí mapy, ale není čistě monotónní. Na menších mapách (20×50 , 70×30) se pohybuje v rozmezí 31,1–42,9 %, přičemž mapa 20×50 dosahuje vyšší hodnoty než větší mapa 70×30 . Na mapě 100×100 činí pokrytí 43,8 % a na mapě 150×150 dosahuje 54,1 %.

Variabilita časů generování je nízká.



Obrázek 8: Místnosti a chodby – průměrný čas generování podle velikosti mapy



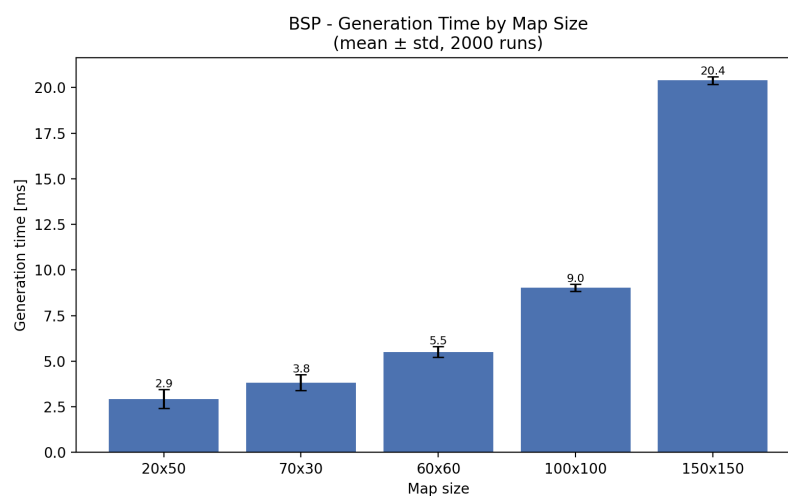
Obrázek 9: Místnosti a chodby – průměrné pokrytí podle velikosti mapy

4.1.2 Binary Space Partitioning

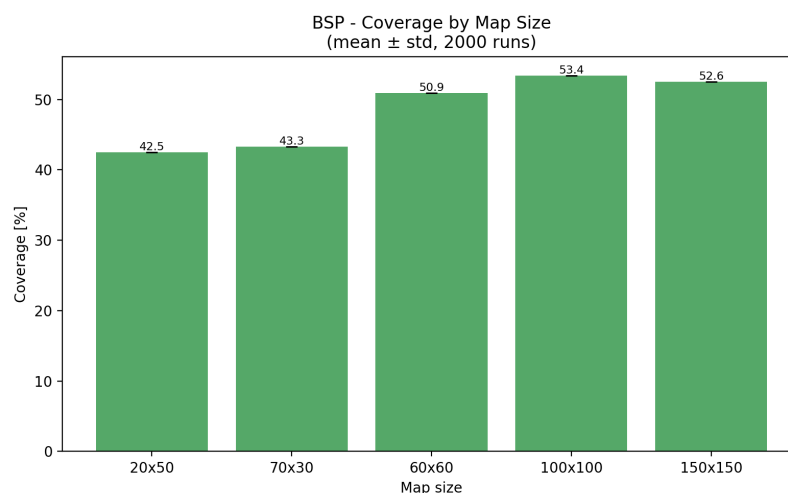
Algoritmus BSP vytváří mapy s dobře definovanými místnostmi a systematickým propojením. Čas generování roste s velikostí map, od přibližně 2,9 ms na mapě 20×50 po 20,4 ms na mapě 150×150. Tento nárůst odpovídá rekurzivní povaze algoritmu, který musí rozdělit celý prostor mapy a následně propojit vzniklé místnosti.

Pokrytí mapy u BSP má celkově rostoucí trend s velikostí mapy. Na mapě 20×50 činí 42,5 %, na mapě 70×30 mírně roste na 43,3 %, na mapě 60×60 stoupá na 50,9 % a na mapě 100×100 dosahuje 53,4 %. Na mapě 150×150 pak pokrytí mírně klesá na 52,6 %.

Variabilita času mezi běhy je u BSP nízká.



Obrázek 10: BSP – průměrný čas generování podle velikosti mapy

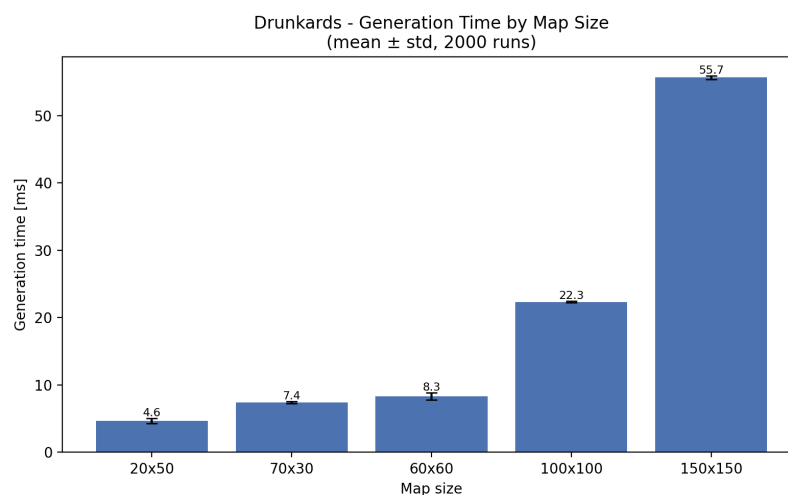


Obrázek 11: BSP – průměrné pokrytí podle velikosti mapy

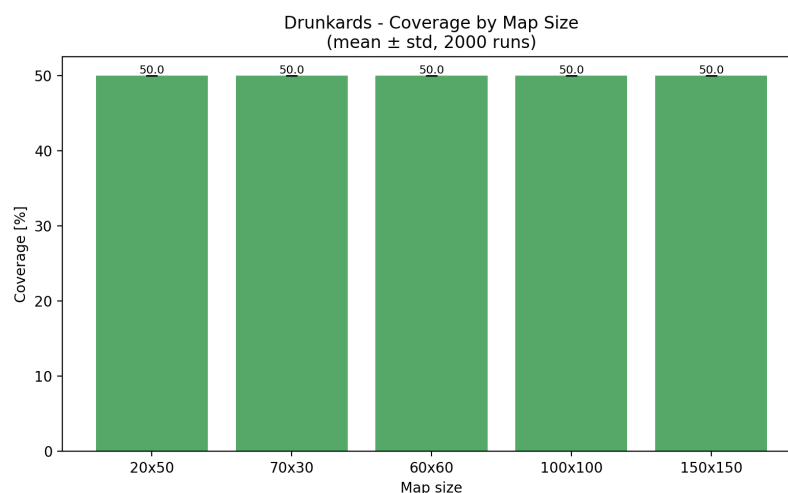
4.1.3 Náhodná procházka

Algoritmus Náhodné procházky se od ostatních metod odlišuje zejména pokrytím mapy. Výsledné pokrytí se ve všech testovaných konfiguracích pohybuje v rozmezí 50,0–50,1 %, což přímo odpovídá implementaci, ve které je cílový počet průchozích dlaždic stanoven pevně. Variabilita pokrytí je tedy prakticky nulová.

Čas generování vykazuje výraznější variabilitu mezi jednotlivými běhy. Na mapě 20×50 se průměrný čas pohybuje kolem 4,6 ms. Na největší mapě 150×150 dosahuje přibližně 55,7 ms. Variabilita časů odpovídá přirozeným výkyvům systému při opakovaném spouštění. Hodnota 50 % je nastavitelný parametr a změnou této konstanty lze dosáhnout libovolného cílového pokrytí.



Obrázek 12: Náhodná procházka – průměrný čas generování podle velikosti mapy



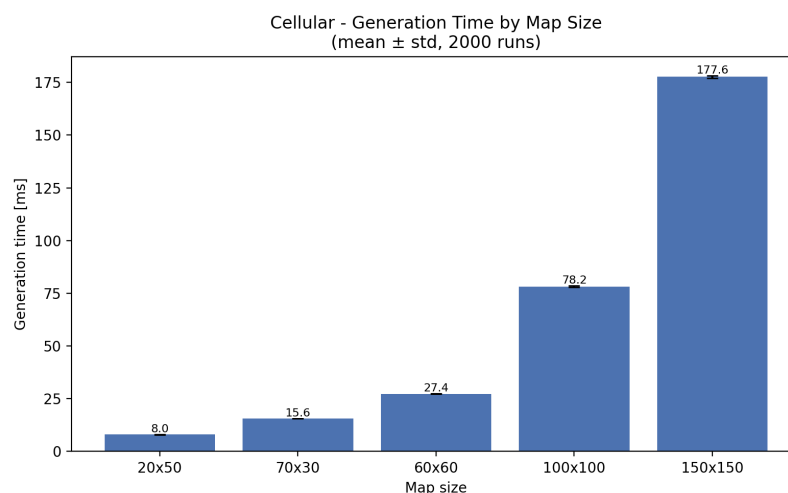
Obrázek 13: Náhodná procházka – průměrné pokrytí podle velikosti mapy

4.1.4 Buněčné automaty

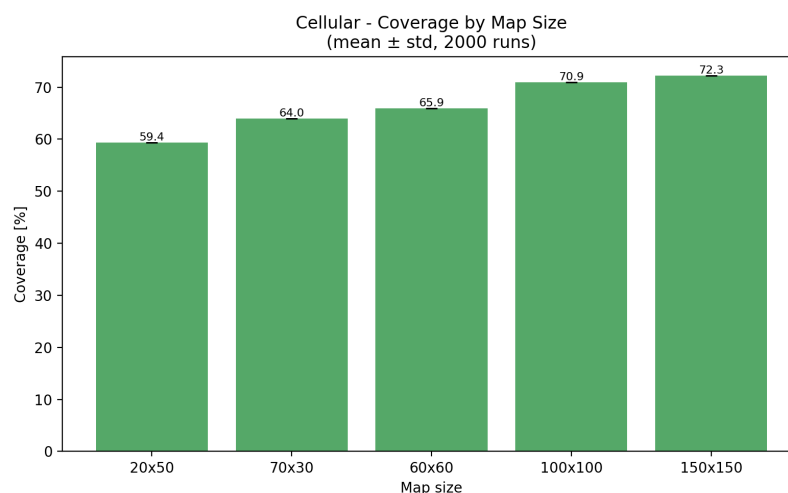
Buněčné automaty jsou z testovaných algoritmů jednoznačně nejpomalejší. Na nejmenší mapě 20×50 trvá generování přibližně 8,0 ms, na mapě 100×100 již 78,2 ms a na největší mapě 150×150 dosahuje průměrný čas přibližně 177,6 ms.

Pokrytí mapy u Buněčných automatů dosahuje vysokých hodnot a s rostoucí velikostí mapy mírně roste. Od přibližně 59,4 % na mapě 20×50 po 72,3 % na mapě 150×150.

Čas generování je mezi běhy prakticky konstantní a směrodatné odchylky jsou velmi malé. Celkový počet operací tak závisí primárně na velikosti mapy.



Obrázek 14: Buněčné automaty – průměrný čas generování podle velikosti mapy

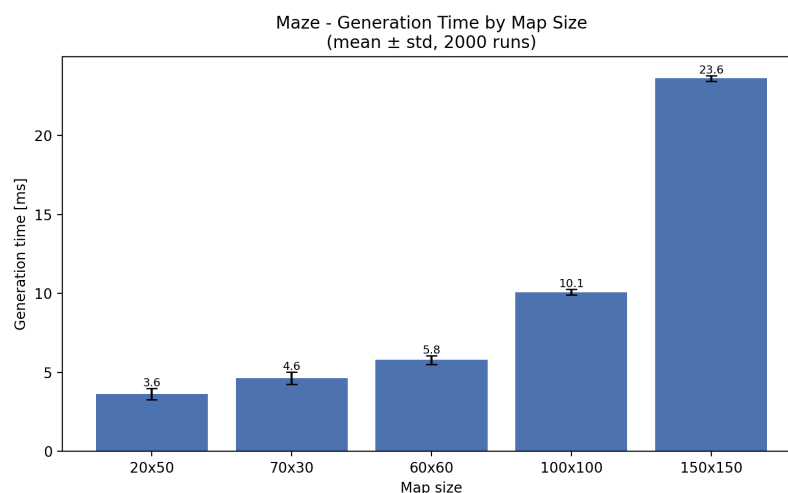


Obrázek 15: Buněčné automaty – průměrné pokrytí podle velikosti mapy

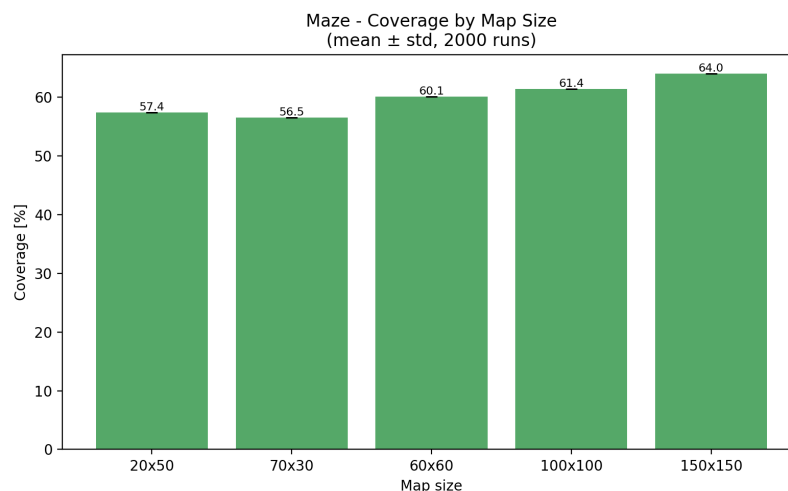
4.1.5 Bludiště

Algoritmus generování Bludiště se vyznačuje zcela deterministickým pokrytím mapy. Hodnota pokrytí je pro danou velikost mapy vždy shodná bez ohledu na konkrétní běh. Na mapě 20×50 činí pokrytí 57,4 %, na mapě 150×150 pak 64,0 %. Nulová variabilita pokrytí vyplývá z principu algoritmu recursive backtracker, který vždy navštíví všechny buňky mřížky a výsledný poměr průchozích dlaždic je dán pouze rozměry mapy a velikostí buněk.

Čas generování je u tohoto algoritmu srovnatelný s BSP. Na nejmenší mapě činí přibližně 3,6 ms, na největší pak 23,6 ms. Variabilita času mezi běhy je nízká.



Obrázek 16: Bludiště – průměrný čas generování podle velikosti mapy



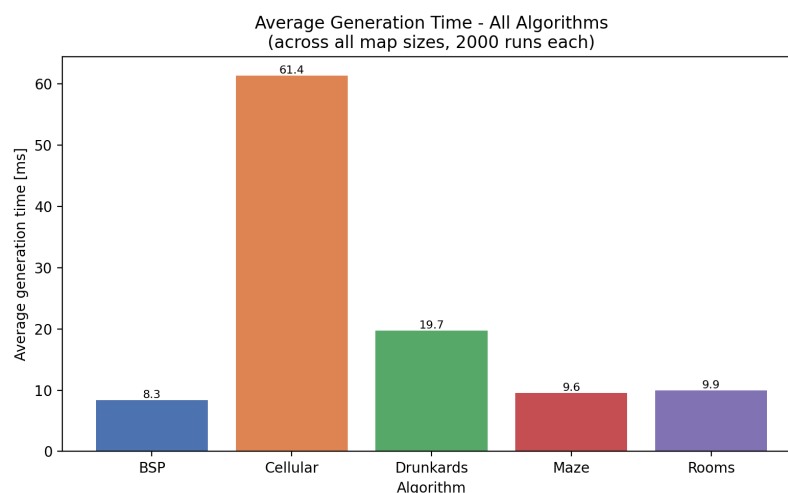
Obrázek 17: Bludiště – průměrné pokrytí podle velikosti mapy

4.2 Srovnání algoritmů

Zobrazené hodnoty představují průměry přes všechny testované velikosti map, pokud není uvedeno jinak (2000 měření na každou kombinaci algoritmu a velikosti, celkem 10 000 měření na algoritmus). Konkrétní hodnoty pro jednotlivé velikosti jsou shrnuty v tabulce [3](#).

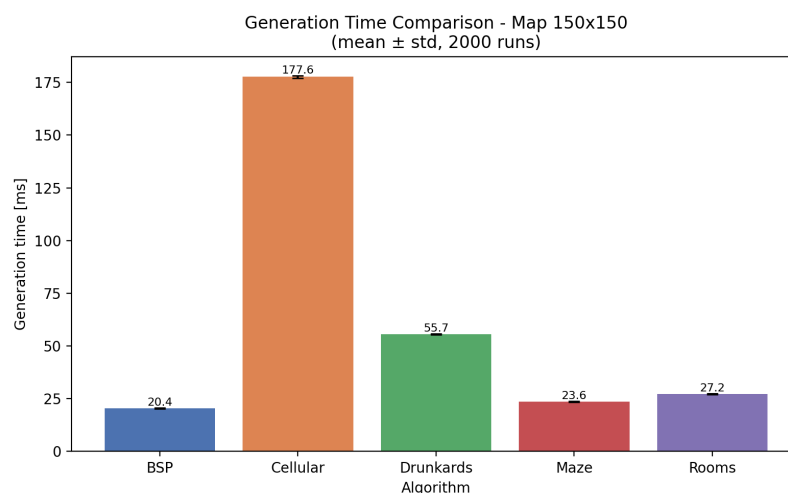
4.2.1 Průměrný čas generování

Graf [18](#) zobrazuje průměrný čas generování algoritmů přes všechny testované velikosti map. Mezi algoritmy jsou patrné výrazné rozdíly. Nejrychlejšími metodami jsou BSP s průměrem 8,3 ms a Bludiště s průměrem 9,6 ms. Místnosti dosahují průměru 9,9 ms. Náhodná procházka je výrazně pomalejší s celkovým průměrem 19,7 ms. Buněčné automaty jsou nejpomalejší s průměrem 61,4 ms, tedy přibližně sedmkrát více než u BSP.



Obrázek 18: Srovnání průměrného času generování všech algoritmů

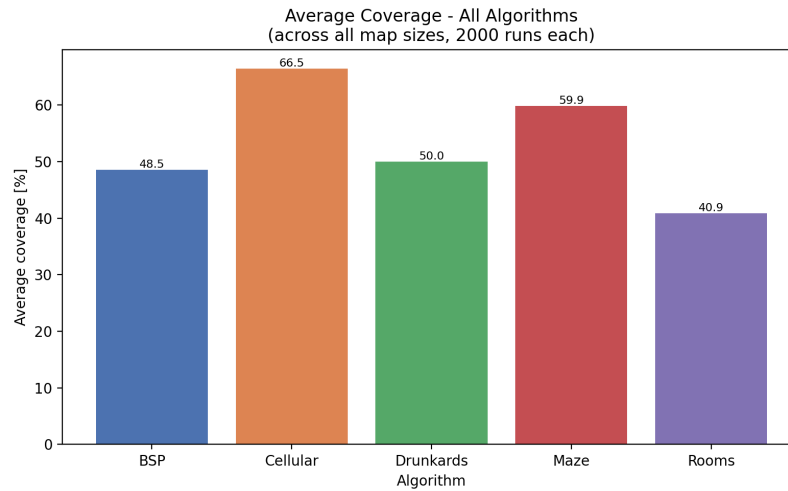
Na největší testované mapě 150×150 jsou absolutní rozdíly ještě výraznější, BSP dosahuje 20,4 ms, Bludiště 23,6 ms, Místnosti 27,2 ms, Náhodná procházka 55,7 ms, zatímco Buněčné automaty přibližně 177,6 ms, jak lze vidět na grafu 19.



Obrázek 19: Srovnání průměrného času generování na mapě 150×150

4.2.2 Průměrné pokrytí

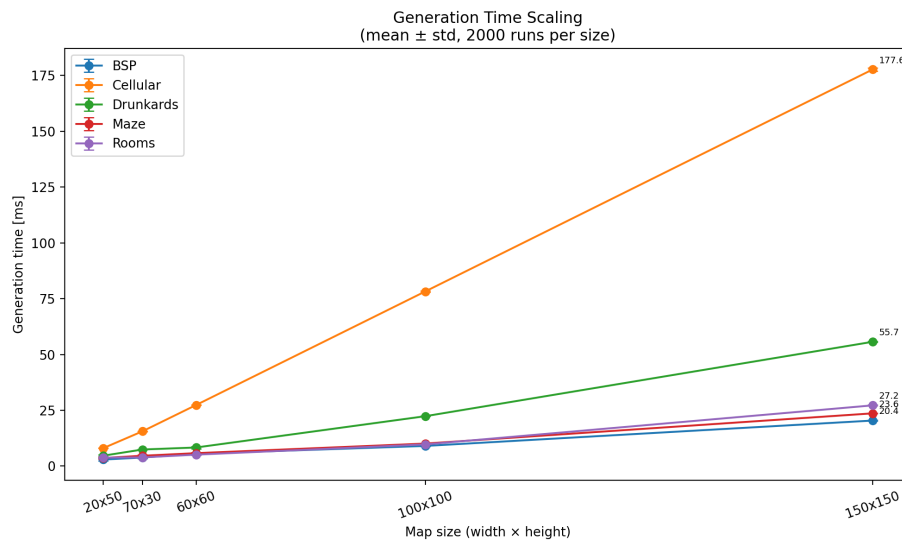
Graf 20 zobrazuje průměrné pokrytí mapy průchozím prostorem. Nejvyššího celkového pokrytí dosahují Buněčné automaty (66,5 %), následované algoritmem Bludiště se stabilním pokrytím 59,9 %. BSP dosahuje celkového průměru 48,5 %, Náhodná procházka má konstrukčně dané pokrytí 50,0 %, které se s velikostí mapy nemění. Místnosti a chodby dosahují nejnižšího celkového průměru, a to 40,9 %, avšak pokrytí výrazně roste s velikostí mapy (viz graf 22).



Obrázek 20: Srovnání průměrného pokrytí všech algoritmů

4.2.3 Škálování s velikostí mapy

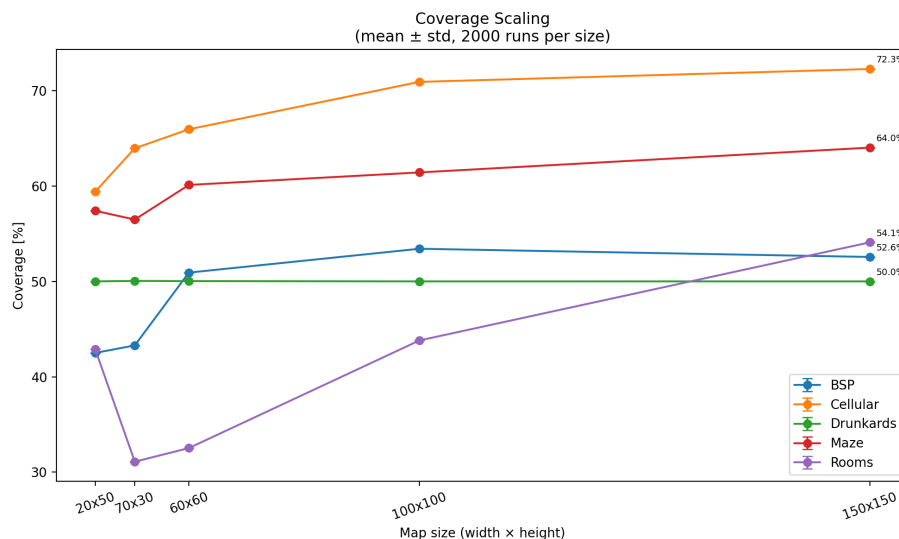
Graf 21 znázorňuje, jak se čas generování mění s rostoucím počtem buněk mapy. BSP a Bludiště vykazují přibližně lineární nárůst času. Algoritmus Místností roste rovněž přibližně lineárně, avšak s vyšší konstantou, protože na velkých mapách generuje výrazně více místností. Náhodná procházka roste rychleji, a to kvůli opakovanému navracení do již odkrytých oblastí. Buněčné automaty vykazují nejstrmější nárůst.



Obrázek 21: Škálování času generování s počtem buněk mapy

Graf 22 ukazuje vývoj pokrytí v závislosti na velikosti mapy. BSP, Buněčné automaty a Místností vykazují celkově rostoucí tendenci, s větší mapou obecně

roste i podíl průchozího prostoru, což je očekávané. U Místností je tento nárůst dán škálováním počtu místností s plochou mapy. Bludiště má mírně rostoucí, ale velmi stabilní pokrytí. Náhodná procházka zůstává konstantní na hodnotě 50 %. V grafu jsou patrné výjimky z tohoto trendu, jejich možné příčiny jsou diskutovány v sekci Diskuse.

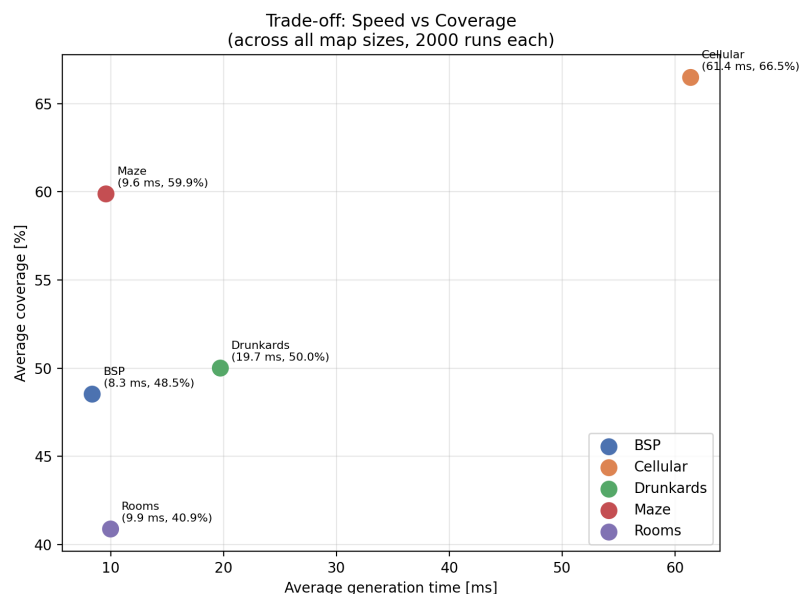


Obrázek 22: Škálování pokrytí s počtem buněk mapy

4.2.4 Kompromis mezi rychlostí a pokrytím

Graf 23 vizualizuje vztah mezi průměrným časem generování a průměrným pokrytím pro jednotlivé algoritmy (oba průměry přes všechny testované velikosti map). Ideální algoritmus by se nacházel v levé horní části grafu, tedy s nízkým časem a vysokým pokrytím.

BSP a Bludiště se nacházejí v oblasti nízkého času a středně vysokého pokrytí, přičemž Bludiště dosahuje vyššího pokrytí (59,9 %) než BSP (48,5 %). Místnosti nabízí srovnatelný čas s nižším celkovým pokrytím. Buněčné automaty dosahují nejvyššího pokrytí ze všech metod, avšak za cenu výrazně vyššího výpočetního času. Náhodná procházka představuje předvídatelné pokrytí 50,0 %, avšak s vyšším časem generování než BSP, Bludiště a Místnosti.

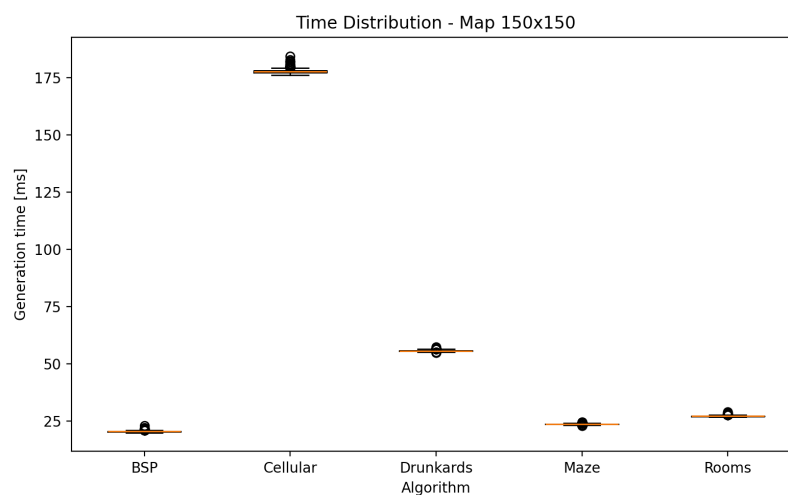


Obrázek 23: Kompromis mezi rychlostí a pokrytím

4.2.5 Rozptyl výsledků na velké mapě

Pro detailnější pohled na variabilitu výsledků jsou uvedeny boxploty času generování a pokrytí na největší testované mapě 150×150 .

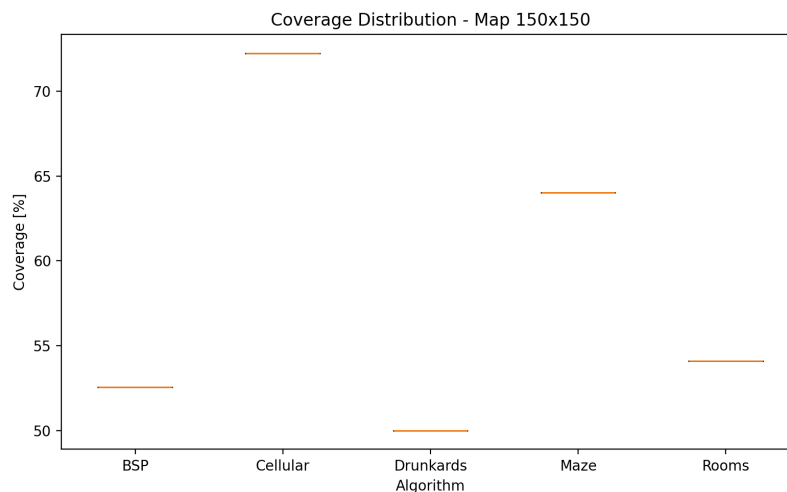
V grafu 24 jsou patrné výrazné absolutní rozdíly v časech generování mezi algoritmy. Variabilita je u všech metod nízká – distribuce jsou úzké s ojedinělými odlehlými hodnotami. Bludiště a Místnosti vykazují nejmenší absolutní rozptyl, zatímco Buněčné automaty mají největší absolutní směrodatnou odchylku.



Obrázek 24: Rozptyl času generování na mapě 150×150

Obrázek 25 ukazuje, že při použití pevného seedu je pokrytí pro všechny

algoritmy zcela deterministické a variabilita pokrytí je nulová napříč všemi metodami.



Obrázek 25: Rozptyl pokrytí na mapě 150×150

4.3 Souhrnná tabulka výsledků

Tabulka 3 shrnuje průměrné hodnoty času generování a pokrytí mapy pro všechny testované kombinace algoritmů a velikostí map.

Algoritmus	20×50	60×60	70×30	100×100	150×150
<i>Průměrný čas generování [ms]</i>					
Rooms	3,81	5,12	3,89	9,72	27,20
BSP	2,93	5,51	3,83	9,03	20,39
Drunkard's	4,65	8,33	7,43	22,34	55,67
Cellular	8,00	27,37	15,58	78,21	177,65
Maze	3,64	5,79	4,63	10,09	23,62
<i>Průměrné pokrytí [%]</i>					
Rooms	42,90	32,53	31,10	43,81	54,09
BSP	42,50	50,92	43,29	53,42	52,56
Drunkard's	50,00	50,03	50,05	50,00	50,00
Cellular	59,40	65,94	63,95	70,91	72,26
Maze	57,40	60,11	56,48	61,42	64,02

Tabulka 3: Průměrný čas generování a pokrytí

4.4 Shrnutí výsledků

Na základě provedených měření lze algoritmy rozdělit do tří skupin podle výpočetní náročnosti. Do první skupiny patří BSP, Bludiště a Místnosti, jejichž časy generování na mapě 150×150 nepřekračují 27,2 ms. Náhodná procházka tvoří

střední kategorii s průměrným časem přibližně 55,7 ms. Buněčné automaty jsou nejpomalejší s průměrným časem přibližně 177,6 ms.

Z hlediska pokrytí mapy se algoritmy rovněž výrazně liší. Buněčné automaty dosahují na velkých mapách nejvyššího pokrytí 72,3 %. Bludiště dosahuje pokrytí v rozmezí 56,5–64,0 % v závislosti na velikosti mapy. BSP dosahuje pokrytí v rozmezí 42,5–53,4 %. Místnosti dosahují na mapě 150×150 přibližně 54,1 %, avšak pokrytí výrazně stoupá s velikostí mapy (od 31,1 % na mapě 70×30). Náhodná procházka má dané pokrytí 50,0 %.

5 Diskuse

5.1 Interpretace výsledků

Naměřené časy generování jsou v souladu s teoretickou časovou složitostí algoritmů. Všechny implementované metody dosahují lineární složitosti $O(n)$, kde $n = w \cdot h$ označuje celkový počet buněk mapy. Rozdíly v rychlosti jsou způsobeny konstantními koeficienty. Buněčné automaty provádějí pro každou buňku $s = 5$ průchodů s vyhodnocením 8 sousedů, tedy přibližně $40 \cdot n$ operací, což odpovídá přibližně osminásobnému rozdílu oproti algoritmům BSP a Bludiště. Interpretovaný jazyk GDScript může vysoký čas Buněčných automatů dále zhoršovat v porovnání s kompilovanými jazyky, avšak primární příčinou zůstává samotná struktura algoritmu. Algoritmus Místností provádí kontrolu překryvů v čase $O(r^2)$, kde $r = O(n/360)$, avšak tento výpočet je v praxi zanedbatelný a dominuje lineární průchod mapou při přidávání stěn. Stochastická povaha Náhodné procházky způsobuje opakované navštěvování odkrytých oblastí a výraznou variabilitu doby generování, i přes lineární průměrnou složitost.

Z hlediska pokrytí se v naměřených výsledcích vyskytují anomálie způsobené tvarem mapy. Mapa 70×30 dosahuje u algoritmů Místností, BSP a Bludiště nižšího pokrytí než mapa 20×50 , přestože celkový počet buněk je větší (2 100 vs. 1 000). Příčinou je tvar mapy – nízký obdélník omezuje rozmístění místností i dělení prostoru. U Místností se do výšky 30 dlaždic nevejde plný počet místností minimální velikosti a chodby zabírají větší podíl plochy. U BSP a Bludiště úzký tvar zmenšuje použitelné oblasti při dělení mřížky, čímž roste podíl stěn a okrajů vůči průchozímu prostoru.

U algoritmu BSP se navíc pokrytí na mapě 100×100 (53,4 %) mírně převyšuje hodnotu dosaženou na mapě 150×150 (52,6 %). Rozdíl je malý, ale naznačuje, že nárůst pokrytí BSP s plochou mapy není zcela lineární.

5.2 Silné a slabé stránky jednotlivých metod

Každý algoritmus vykazuje specifické silné a slabé stránky, které ovlivňují jeho vhodnost pro různé typy her.

Algoritmus BSP má dobrou rovnováhu mezi rychlostí generování, pokrytím mapy a strukturovaností výsledku. Generované mapy obsahují jasně oddělené místnosti propojené chodbami. Nevýhodou je poměrně pravidelný vzhled výsledných map.

Algoritmus Místností dosahuje na velkých mapách výrazně vyššího pokrytí a zároveň nabízí přirozenější rozložení prostoru.

Buněčné automaty generují přirozeně působící jeskynní struktury, které se nejvíce odlišují od klasického dungeonového prostředí. Dosahují nejvyššího pokrytí ze všech testovaných metod. Hlavní nevýhodou je výrazně vyšší výpočetní náročnost a možný vznik izolovaných oblastí, které nejsou vzájemně průchozí. Důsledkem těchto izolovaných oblastí je, že reportované pokrytí zahrnuje i nepřístupné části mapy. Skutečné přístupné pokrytí je proto mírně nižší, než jakou

hodnotu udává měřená metrika.

Náhodná procházka vytváří organicky tvarované chodby s předvídatelným pokrytím. Konstantní pokrytí může být výhodou v situacích, kdy je požadována přesná kontrola nad poměrem průchozího prostoru. Nevýhodou je nižší strukturovanost výsledného prostředí.

Algoritmus Bludiště byl do implementace zahrnut primárně pro demonstraci odlišného přístupu k procedurálnímu generování. Generuje dokonalé bludiště. Z hlediska dungeonových map je však méně vhodný, protože vytváří převážně úzké chodby bez většího otevřeného prostoru.

5.3 Zkušenosti z implementace

U většiny metod byla průchodnost mapy zajištěna již principem algoritmu, tedy explicitní propojování místností u Místností a BSP, návštěva všech buněk u Bludiště a spojitá chůze u Náhodné procházky. U Buněčných automatů však mohou po simulaci vzniknout oddělené nepřístupné kapsy. Pro umístění hráče tak bylo nutné nejprve pomocí BFS identifikovat největší souvislou oblast, což bylo nakonec implementováno jako společná část všech algoritmů.

Velikost hráče ovlivnila i návrh všech generovacích algoritmů. Bylo nutné zajistit, aby chodby byly široké alespoň 2 dlaždice, jinak by byla mapa pro hráče neprůchozí. Toto omezení se promítlo například do šířky L-chodeb u BSP a Místností, do vykreslování oblastí 2×2 v jednotlivých krocích Náhodné procházky a do velikosti buněk 2×2 u algoritmu Bludiště.

Úprava algoritmu Místností, při které byl pevný počet místností nahrazen škálováním úměrným ploše mapy, vedla k výraznému zlepšení pokrytí na velkých mapách. Další významná úprava proběhla v rámci BSP, kdy místo fixních 4 průchodů do hloubky byl algoritmus pozměněn na rekurzivní dělení, které pokračuje, dokud by výsledné podoblasti nebyly menší než $\text{MIN_SIZE} \times 2$.

Ladění parametrů Buněčných automatů, v rámci počtu simulačních kroků, mělo značný dopad na výsledek. Při 3 krocích zůstávaly v mapě četné izolované buňky, při 10 krocích se struktury výrazně vyhladily a pokrytí vzrostlo, avšak za cenu vyššího výpočetního času a velkého prázdného prostoru bez struktury. Hodnota 5 kroků představuje kompromis mezi kvalitou výsledných struktur a časovou náročností. Podobně parametr `TARGET_COVERAGE` u Náhodné procházky přímo určuje podíl průchozího prostoru, vyšší hodnota prodlužovala dobu generování a struktura byla opět velmi souvislá.

Celkově tyto zkušenosti ukazují, že i drobná změna parametrizace může zásadně ovlivnit výsledné vlastnosti generovaného prostředí, jeho pokrytí, strukturu i čas generování.

5.4 Souvislost s předchozím výzkumem

Porovnáním efektivity PCG algoritmů pro generování 2D map se zabývala rovněž práce [12], která měřila čas generování a spotřebu výpočetních zdrojů kombinací

algoritmů pro umísťování místností (Random Room Placement, BSP) a propojování chodbami (Random Point Connect, Drunkard's Walk, BSP Corridors). Oproti tomuto přístupu se předkládaná práce zaměřuje na pět samostatných generovacích metod (včetně Buněčných automatů a generování Bludiště) a jako metriku používá vedle času generování také pokrytí mapy průchozím prostorem. Obě práce se však shodují v závěru, že BSP patří mezi nejefektivnější přístupy z hlediska času generování.

5.5 Možnosti dalšího rozšíření práce

Prvním směrem rozšíření je zahrnutí strukturálních metrik. Pokrytí mapy průchozím prostorem je snadno měřitelná a objektivní metrika, avšak sama o sobě nevypovídá o strukturální kvalitě generovaného prostředí. Dvě mapy se stejným pokrytím se mohou zásadně lišit z hlediska hratelnosti. Například BSP a Náhodná procházka dosahují na mapě 60×60 prakticky shodného pokrytí (50,9 a 50,0 %), přesto jsou výsledné struktury zcela odlišné: BSP vytváří jasně oddělené místnosti propojené chodbami, zatímco Náhodná procházka generuje organicky tvarované chodby bez pevné struktury.

Relevantnějšími doplňkovými metrikami by mohly být například průměrná délka nejkratší cesty mezi dvěma náhodně vybranými průchozími dlaždicemi (měřitelná pomocí BFS), podíl slepých uliček nebo průměrný počet prostorově oddělených místností. Tyto metriky by umožnily kvantitativně zachytit rozdíly, které se v současném měření projevují pouze vizuálně, a lépe posoudit vhodnost jednotlivých metod pro konkrétní typy herního prostředí.

Dalším směrem je kombinace více metod v rámci jedné mapy, například vyplnění BSP místností buněčnými automaty pro dosažení přirozenějšího vzhledu. Rozšířením by mohlo být také přidání sémantických pravidel pro rozmístění herních prvků, například zajištění, aby klíčové předměty byly umístěny v dostatečné vzdálenosti od startu hráče.

V neposlední řadě by bylo možné implementovat adaptivní generování, které by přizpůsobovalo parametry algoritmů na základě postupu hráče hrou.

Závěr

Cílem této bakalářské práce bylo představit, implementovat a porovnat vybrané metody procedurálního generování dungeonových map ve 2D hrách.

V teoretické části práce byly popsány základní principy procedurálního generování obsahu, jeho historický vývoj a přehled vybraných metod včetně jejich výhod a omezení. Na základě provedené rešerše bylo vybráno pět algoritmů reprezentujících různé přístupy ke generování map: Náhodné umístění místností, Binary Space Partitioning, Náhodná procházka (Drunkard's Walk), Buněčné automaty a generování Bludiště.

Všech pět algoritmů bylo implementováno v herním enginu Godot s využitím skriptovacího jazyka GDScript. Systém byl navržen modulárně s unifikovaným rozhraním, které umožňuje snadné přidávání nových generovacích metod. Součástí aplikace je automatizovaný benchmarkový režim, který zajišťuje objektivní a opakovatelné měření výkonu jednotlivých algoritmů.

Experimentální měření bylo provedeno na pěti velikostech map v 2000 opakováních pro každou konfiguraci se seedem 05102024. Výsledky ukázaly výrazné rozdíly mezi metodami. Algoritmy BSP a Bludiště se ukázaly jako nejrychlejší s časy 20,4–23,6 ms na mapě 150×150. Algoritmus Místností dosahuje pokrytí přibližně 54,1 % na největší mapě. Buněčné automaty generují mapy s nejvyšším pokrytím (72,3 %), avšak s výrazně vyšším výpočetním časem, a to přibližně 177,6 ms na mapě 150×150. Náhodná procházka poskytuje konstrukčně dané pokrytí 50,0 %, avšak s vyšším časem generování, přibližně 55,7 ms na největší mapě.

Práce ukázala, že volba generovací metody závisí na konkrétních požadavcích hry, jelikož výsledná struktura je různá. Pro klasické dungeonové prostředí jsou vhodné algoritmy BSP a Místností, pro jeskynní struktury jsou ideální Buněčné automaty a Náhodná procházka a pro labyrinty algoritmus Bludiště.

Mezi hlavní omezení práce patří testování výhradně v prostředí Godot s jazykem GDScript a měření na jednom hardwarovém zařízení (MacBook Air 2025, M4, 24 GB RAM). Benchmarkové měření probíhalo s pevným seedem, čímž jsou výsledky plně reprodukovatelné. Statistické závěry jsou podloženy 2000 měřeními na každou konfiguraci.

Conclusions

The aim of this bachelor’s thesis was to present, implement, and compare selected methods of procedural dungeon map generation in 2D games.

The theoretical part described the basic principles of procedural content generation, its historical development, and an overview of selected methods, including their advantages and limitations. Based on the literature review, five algorithms representing different approaches to map generation were selected: Random room placement, Binary Space Partitioning, Drunkard’s walk, Cellular automata, and Maze generation.

All five algorithms were implemented in the Godot game engine using the GDScript scripting language. The system was designed in a modular fashion with a unified interface that allows easy addition of new generation methods. The application includes an automated benchmark mode that ensures objective and repeatable performance measurements.

Experimental measurements were conducted on five map sizes with 2000 runs per configuration using a fixed seed (05102024). The results revealed significant differences between the methods. BSP and Maze proved to be the fastest algorithms, with generation times of 20.4–23.6 ms on a 150×150 map. The Rooms algorithm achieves approximately 54.1 % coverage on the largest map. Cellular automata generate maps with the highest coverage (72.3 %), but at the cost of significantly higher computation time of approximately 177.6 ms on a 150×150 map. Drunkard’s walk provides a structurally fixed coverage of 50.0 %, but with a higher generation time of approximately 55.7 ms on the largest map.

The thesis demonstrated that the choice of generation method depends on the specific requirements of the game. BSP and Rooms are suitable for classic dungeon environments, cellular automata and drunkard’s walk for cave-like structures, and the Maze algorithm for labyrinthine layouts.

Limitations of this work include testing exclusively in the Godot environment with GDScript and measurements on a single hardware device (MacBook Air 2025, M4, 24 GB RAM). The benchmark measurements were conducted with a fixed seed, ensuring full reproducibility of results. The statistical conclusions are supported by 2000 measurements per configuration.

A Odkaz na repozitář projektu

Zdrojový kód aplikace je dostupný v repozitáři na platformě GitHub:

<https://github.com/Jim-23/ProceduralGenerationBP>

B Obsah elektronických dat

Elektronická data práce jsou odevzdána ve formě ZIP archivu celého projektu s následující strukturou:

src/

Zdrojové soubory projektu a README.md s instrukcemi pro spuštění aplikace.

text/

Adresář s textem bakalářské práce.

Seznam zkratek

2D Two-Dimensional

3D Three-Dimensional

ASCII American Standard Code for Information Interchange

BBC Micro British Broadcasting Corporation Microcomputer

BFS Breadth-First Search

BSP Binary Space Partitioning

CSV Comma-Separated Values

DFS Depth-First Search

PCG Procedural Content Generation

RAM Random Access Memory

UI User Interface

Literatura

- [1] Shaker, Noor; Togelius, Julian; Nelson, Mark J. *Procedural Content Generation in Games: A Textbook and an Overview of Current Research*. 2016. 247 s. ISBN 978-3-319-42714-0.
- [2] Linden, Roland; Lopes, Ricardo; Bidarra, Rafael. Procedural Generation of Dungeons. *Computational Intelligence and AI in Games, IEEE Transactions on*. 2014, roč. 6, s. 78–89. Dostupný také z: <http://dx.doi.org/10.1109/TCIAIG.2013.2290371>.
- [3] Schuller, Dan. *Dive into beneath Apple Manor: A pre-rogue roguelike*. 2021. Dostupný z: <https://howtomakeanrpg.com/r/l/g/apple-manor.html>.
- [4] Wikipedia Contributors. *Beneath Apple Manor* — *Wikipedia, The Free Encyclopedia*. 2024. Dostupný z: https://en.wikipedia.org/w/index.php?title=Beneath_Apple_Manor&oldid=1260179812.
- [5] Procedural Content Generation Wiki. *Rogue* [online]. 2016 [cit. 2024-5-22]. Dostupný z: <http://pcg.wikidot.com/pcg-games:rogue>.
- [6] Contributors, Wikipedia. *Rogue (video game)* — *Wikipedia, The Free Encyclopedia*. 2011. Dostupný z: [https://en.wikipedia.org/w/index.php?title=Rogue_\(video_game\)&oldid=1337873899](https://en.wikipedia.org/w/index.php?title=Rogue_(video_game)&oldid=1337873899).
- [7] Contributors, Wikipedia. *Elite (video game)* — *Wikipedia, The Free Encyclopedia*. 2005. Dostupný z: [https://en.wikipedia.org/wiki/Elite_\(video_game\)](https://en.wikipedia.org/wiki/Elite_(video_game)).
- [8] Gumin, Maxim. *WaveFunctionCollapse*. 2016. Dostupný z: <https://github.com/mxgmn/WaveFunctionCollapse>.
- [9] *Godot Engine Documentation*. 2024. Dostupný z: <https://docs.godotengine.org>.
- [10] *GDScript Reference*. 2024. Dostupný z: <https://docs.godotengine.org/en/stable/tutorials/scripting/gdsript/>.
- [11] Pixel_Poem. *2D Pixel Dungeon Asset Pack*. 2023. Dostupný z: <https://pixel-poem.itch.io/dungeon-assetpack>.
- [12] Monaghan, Zina. *Comparing Procedural Content Generation Algorithms for Creating Levels in Video Games*. 2019. Dostupný také z: <https://arrow.tudublin.ie/cgi/viewcontent.cgi?article=1189&context=scschcomdis>.